Cross-Lingual Dependency Parsing for Low-Resource Bhojpuri via Parallel Bottleneck Adapters and Syntax-Aware Transfer

Thesis submitted in fulfillment of the requirements

for the M.Tech Project of

Fifth Year IDD

by

Kolgane Sanskruti Sanjay

Roll No. 21074015

Under the guidance of

Dr. Anil Kumar Singh

Department of Computer Science and Engineering

Department of Computer Science and Engineering

INDIAN INSTITUTE OF TECHNOLOGY (BHU) VARANASI

Varanasi 221005, India

May 2026

Dedicated to

My parents, teachers, family and friends

Declaration

I certify that, the work contained in this report is original and has been done by myself and the general supervision of my supervisor. The work has not been submitted for any project. Whenever I have used materials (data, theoretical analysis, results) from other sources, I have given due credit to them by citing them in the text of the thesis and giving their details in the references. Whenever I have quoted written materials from other sources, I have put them under quotation marks and given due credit to the sources by citing them and giving required details in the references.

Place: IIT (BHU) Varanasi
Date:

Kolgane Sanskruti Sanjay
IDD Student
Department of Computer Science and Engineering,
Indian Institute of Technology (BHU) Varanasi,
Varanasi, INDIA 221005.

CERTIFICATE BY THE SUPERVISOR

It is certified that the above statement made by the student is correct to the best of my/our knowledge.

Place: IIT (BHU) Varanasi
Date:

Dr. Anil Kumar Singh
Department of Computer Science and Engineering,
Indian Institute of Technology (BHU) Varanasi,
Varanasi, INDIA 221005.

Signature of Head of Department

Certificate

This is to certify that the work contained in this report entitled **"Cross-Lingual Dependency Parsing for Low-Resource Bhojpuri via Parallel Bottleneck Adapters and Syntax-Aware Transfer"** being submitted by **Kolgane Sanskruti Sanjay (Roll No. 21074015)**, carried out in the Department of Computer Science and Engineering, Indian Institute of Technology (BHU) Varanasi, is a bona fide work of our supervision.

Dr. Anil Kumar Singh
Department of Computer Science and Engineering,
Indian Institute of Technology (BHU) Varanasi,
Varanasi, INDIA 221005.

Place: IIT (BHU) Varanasi

Date:

COPYRIGHT TRANSFER CERTIFICATE

Title of the Thesis: Cross-Lingual Dependency Parsing for Low-Resource Bhojpuri via Parallel Bottleneck Adapters and Syntax-Aware Transfer

Name of the Student: Kolgane Sanskruti Sanjay

COPYRIGHT TRANSFER

The undersigned hereby assigns to the **Indian Institute of Technology (Banaras Hindu University), Varanasi** all rights under copyright that may exist in and for the above thesis submitted for the award of the **Integrated Dual Degree (B.Tech. + M.Tech.)** in **Computer Science and Engineering**.

Place: IIT (BHU) Varanasi Signature of the student
Date:

Note: However, the author may reproduce or authorize others to reproduce material extracted verbatim from the thesis or derivative of the thesis for author's personal use provided that the source and the Institute's copyright notice are indicated.

Acknowledgments

I would like to express my sincere gratitude to my thesis supervisor, **Dr. Anil Kumar Singh**, for his invaluable guidance, continuous motivation, and valuable feedback throughout the course of this work. His deep expertise in computational linguistics and natural language processing, and his commitment to the development of language technology for Indic languages, shaped the direction of this research in essential ways. I am especially thankful for the high-quality parallel Hindi–Bhojpuri dataset he provided, which forms the empirical backbone of this entire thesis.

I would like to thank the Faculty and Staff of the Computer Science and Engineering Department of IIT (BHU) Varanasi for their encouragement during the learning process. I thank the members of the NLTM (Natural Language and Text Mining) laboratory at IIT (BHU) for useful discussions and a stimulating research environment. I am grateful to the National Supercomputing Mission for providing access to **Param Shivay**, the high-performance computing facility at IIT (BHU), on which the majority of the experiments reported in this thesis were conducted.

I am also grateful to the developers of the **Universal Dependencies** project and the creators of the **Bhojpuri Hindi Treebank** (BHTB) for making their resources publicly available. The pretrained **XLM-RoBERTa** and **Trankit** models from Meta AI and the University of Oregon respectively provided the multilingual backbone on which this work builds.

I thank my friends and fellow students for discussion and encouragement during the difficult stages of this thesis. Lastly, I thank my family for their motivation, encouragement and support that helped me throughout this journey.

Place: IIT (BHU) Varanasi **Kolgane Sanskruti Sanjay**
Date:

Abstract

Dependency parsing is a fundamental task in natural language processing that structures sentences as directed trees encoding grammatical relations. For low-resource languages such as Bhojpuri—an Indo-Aryan language spoken by approximately 50 million people, primarily in the Indian states of Bihar and Uttar Pradesh—building accurate dependency parsers is severely hampered by the scarcity of annotated treebanks.

This thesis investigates **cross-lingual transfer** from Hindi to Bhojpuri for Universal Dependency parsing, exploiting the close linguistic kinship between the two languages. We present a progression of five systems (A, F, G, H, K) organised across two complementary evaluation parts. **Part 1** targets the aligned-data annotation schema using 30,966 parallel Hindi–Bhojpuri sentences with transferred annotations, evaluated on a held-out development set. **Part 2** targets the official BHTB gold test set (357 sentences) annotated following UD guidelines, addressing the critical schema mismatch between the Hindi–Bhojpuri aligned data and the external benchmark.

The baseline **System A** uses the Trankit XLM-RoBERTa pipeline trained on Hindi in a zero-shot transfer setting, achieving 52.78% UAS/35.36% LAS on the BHTB test set. **System F** fine-tunes Trankit on the aligned-data 30,966 Bhojpuri sentences with warm-starting from the Hindi checkpoint; it achieves strong performance on the Hindi–Bhojpuri aligned data but suffers on BHTB due to the annotation schema mismatch.

To improve cross-lingual representation learning, we propose a **Parallel Adapter architecture** (Systems G and H) that freezes the shared XLM-RoBERTa backbone and trains only small language-specific Pfeiffer-style bottleneck adapters ($r = 64$, $\sim$100K parameters each) alongside biaffine scoring heads. **System G** adds a mean-squared-error positional alignment loss, reaching 55.70%/50.02% on the development set. **System H (SACT)** replaces the MSE with three targeted components: content-word cosine alignment, arc-distribution KL distillation, and Cross-lingual Tree Supervision (CTS), achieving **55.85%/50.08%**—the highest development set LAS among all systems, winning Part 1.

For Part 2, **System K (UD-Bridge)** addresses the schema mismatch by training exclusively on UD-style data: it generates silver UD labels for Bhojpuri using System A's predictions, concatenates them with Hindi HDTB gold data, and trains a Trankit pipeline. By operating entirely in the UD annotation space, System K

achieves **54.27%/36.70%** on BHTB, surpassing System A by +1.49 **uas**/+1.34 **las** and establishing a new state-of-the-art on the official Bhojpuri benchmark. This demonstrates that self-training with UD-consistent silver data can bridge the gap between rich parallel data and gold-standard evaluation.

Together, these systems demonstrate that **parameter-efficient adapter modules combined with syntax-aware cross-lingual objectives** offer a practical and effective approach to dependency parsing for low-resource Indic languages, while schema-aware self-training provides a complementary path to maximising performance on external benchmarks.

Keywords: dependency parsing, cross-lingual transfer, low-resource NLP, Bhojpuri, Hindi, bottleneck adapters, XLM-RoBERTa, biaffine attention, Universal Dependencies, self-training.

Contents

1 Introduction **1**

1.1 Problem Statement 1

1.2 Motivation 1

1.3 Focus on Low-Resource Languages and Associated Challenges 2

1.4 Objectives 3

1.5 Two-Part Evaluation Framework 4

1.6 Contributions and Organisation of Thesis 4

1.6.1 Contributions 4

1.6.2 Organisation of Thesis 5

2 Background and Preliminaries **7**

2.1 Overview of NLP Analysis Tasks 7

2.1.1 Tokenisation 7

2.1.2 Part-of-Speech Tagging 7

2.1.3 Dependency Parsing 7

2.2 Bhojpuri: A Low-Resource Indo-Aryan Language 8

2.3 A Brief History of Dependency Parsing 8

2.4 Universal Dependencies (UD) Framework 9

2.4.1 The Three UD Layers 9

2.4.2 The CoNLL-U File Format 10

2.4.3 Why UD Matters for This Thesis 10

2.5 Biaffine Dependency Parsing 11

2.5.1 Contextual Encoding 11

2.5.2 Role-Specific Projections 11

2.5.3 Bilinear Arc Scoring 11

2.5.4 Per-Word Head Distributions 12

2.5.5 Label Scoring 12

2.5.6 Decoding to a Tree 12

2.5.7 Training Loss 13

2.5.8 Evaluation Metrics 13

2.6 Pretrained Multilingual Language Models 13

2.6.1 XLM-RoBERTa 13

2.6.2 Trankit . . . 13
2.7 Adapter Modules for Parameter-Efficient Transfer . . . 14
2.7.1 Motivation and Background . . . 14
2.7.2 Pfeiffer Adapter Architecture . . . 14
2.7.3 Parameter Count . . . 14
2.7.4 Why Adapters Avoid Catastrophic Forgetting . . . 15
2.7.5 Composition and Modularity . . . 15
2.8 Catastrophic Forgetting . . . 15
2.9 Key Terms . . . 15

3 Related Works 17
3.1 Background of Sentence-Level Analysis in NLP . . . 17
3.2 Cross-lingual Dependency Parsing . . . 17
3.3 Parameter-Efficient Transfer Learning . . . 18
3.4 Low-Resource NLP for Indic Languages . . . 19
3.5 Self-Training and Knowledge Distillation . . . 19
3.6 Summary and Research Gap . . . 20

4 Proposed Work and Methodology 21
4.1 Overview of the System Hierarchy . . . 21
4.2 System A: Zero-Shot Cross-lingual Baseline . . . 21
4.3 System F: Warm-Start Fine-tuning on Aligned Data . . . 22
4.4 System G: Parallel Adapters with Positional Alignment . . . 23
4.4.1 Motivation . . . 23
4.4.2 Architecture . . . 23
4.4.3 Training Objective . . . 24
4.5 System H: Syntax-Aware Cross-lingual Transfer (SACT) . . . 24
4.5.1 Motivation . . . 24
4.5.2 Component 1: Content-Word Cosine Alignment . . . 24
4.5.3 Component 2: Arc-Distribution KL Distillation . . . 25
4.5.4 Component 3: Cross-lingual Tree Supervision (CTS) . . . 25
4.5.5 Combined SACT Objective . . . 25
4.6 System K: UD-Bridge via Silver Self-Training . . . 25
4.6.1 Motivation: The Schema Mismatch Problem . . . 25
4.6.2 Architecture . . . 26
4.6.3 Silver Label Generation Pipeline . . . 26
4.6.4 Why UD-Bridge Works . . . 26
4.7 Training Procedures and Algorithmic Details . . . 27
4.7.1 System F: Naive Warm-Start Fine-Tuning . . . 27
4.7.2 System G: Parallel Adapters with MSE Alignment . . . 28

4.7.3 System H: SACT Training Loop 28
4.7.4 System K: UD-Bridge Training Pipeline 29
4.7.5 Computational Complexity Summary 29
4.8 Summary of Pipelines and Methodology 30

5 Experimental Setup and Results 31
5.1 Experimental Environment 31
5.1.1 Hardware 31
5.1.2 Software Environment 31
5.2 Datasets 31
5.2.1 Hindi–Bhojpuri Aligned Parallel Corpus 31
5.2.2 Data Split 32
5.2.3 Hindi HDTB 32
5.2.4 BHTB Test Set 32
5.3 Experimental Groups and Configurations 32
5.3.1 Part 1: Aligned-Data Schema (Systems F, G, H) 32
5.3.2 Part 2: BHTB UD Schema (Systems A, K) 32
5.4 Hyperparameters 33
5.5 Results and Quantitative Analysis 33
5.5.1 Part 1 Results: Aligned-Data Schema 33
5.5.2 Part 2 Results: BHTB UD Benchmark 34
5.5.3 Cross-Part Analysis: Why the Gap Exists 35
5.5.4 Epoch-by-Epoch Learning Curves 35
5.5.5 Ablation Study 36
5.5.6 Comparison with Published Trankit Hindi Results 36
5.5.7 Per-Relation Error Analysis 37
5.5.8 Statistical Considerations on the BHTB Test Set 38

6 Discussion 39
6.1 Interpretation of Results 39
6.1.1 Part 1: Novel Architectures 39
6.1.2 Part 2: Schema-Aware Self-Training 39
6.1.3 The Annotation Schema Mismatch 40
6.2 Strengths and Weaknesses of the Approach 40
6.2.1 Strengths 40
6.2.2 Weaknesses 41
6.3 Practical Implications 41
6.4 Generalisability to Other Low-Resource Languages 41
6.5 Qualitative Error Analysis 41
6.6 Limitations and Threats to Validity 42

6.7 Practical Deployment Considerations 43

7 Conclusion and Future Work 44

7.1 Summary of Contributions 44
7.2 Key Takeaways 45
7.3 Methodological Considerations 45
7.4 Future Work 45
7.4.1 Gold Bhojpuri Data Collection 45
7.4.2 Synthetic Bhojpuri Dataset Augmentation 45
7.4.3 Multi-layer Adapters 46
7.4.4 Iterative Self-Training 46
7.4.5 Extension to Other Low-Resource Indic Languages 46
7.5 Broader Reflections and Conclusion 46

A Implementation Details 47

A.1 Code Organisation 47
A.2 Data Files 47
A.3 Reproducibility Checklist 48
A.4 Hyperparameter Sensitivity 48

B Universal Dependencies Relation Inventory 50

B.1 Core Argument Relations 50
B.2 Nominal Modifier Relations 51
B.3 Coordination, Auxiliary, and Function Relations 51
B.4 Bhojpuri-Specific Annotation Considerations 51

C Sample Parses and Qualitative Examples 53

C.1 Example 1: A Successfully Parsed Sentence 53
C.2 Example 2: Schema-Mismatch Failure 53
C.3 Example 3: Persistent Failure on Clausal Complement 54

D Detailed Loss Function Derivations 55

D.1 Content-Word Cosine Alignment 55
D.2 Arc-Distribution KL Distillation 56
D.3 Cross-lingual Tree Supervision 56
D.4 Combined Objective 57

E Detailed Dataset Statistics 58

E.1 Sentence Length Distributions 58
E.2 UPOS Tag Distributions 58
E.3 Dependency Relation Frequencies in BHTB 59

E.4 Hindi–Bhojpuri Aligned Corpus Source Composition 60
E.5 Inter-Annotator Agreement Considerations 60

F Comparison with Published Indic UD Parsing Results 61
F.1 Hindi Parsing Benchmarks 61
F.2 Bhojpuri / Low-Resource Indic Parsing 61
F.3 Why Cross-Lingual UD Numbers Look Lower 62

G Glossary of Key Terms 63

List of Figures

4.1 System G: Parallel adapter architecture with MSE positional alignment. 23
4.2 System K: UD-Bridge via silver self-training. System A generates UD-consistent silver labels for Bhojpuri, which are concatenated with Hindi HDTB gold data to train a UD-native Trankit parser. 27

List of Tables

1 Symbol Description xvi

4.1 Summary of the five systems and their part assignments. 21
4.2 Computational and parameter cost summary. 29
4.3 Summary of training data and evaluation targets for each system. . . 30

5.1 Part 1 experimental configurations. 32
5.2 Part 2 experimental configurations. 33
5.3 Hyperparameters for Systems G and H. 33
5.4 Part 1 results (Dev set). Systems F, G, H compete on the aligned-data annotation schema. Best values are **bold**. 33
5.5 Part 2 results (BHTB test set). Systems A and K compete on the UD gold benchmark. Best values are **bold**. 34
5.6 Cross-part comparison showing the annotation schema mismatch effect. Part 1 systems perform well on Dev (aligned-data schema) but poorly on BHTB (UD schema), while Part 2 systems target BHTB directly. 35
5.7 Development set LAS across training epochs for Systems G and H. . 35
5.8 Ablation study on System H components (Dev set LAS, best epoch). 36
5.9 Comparison of System A's reproduced Hindi-HDTB scores against published Trankit numbers. Both use the XLM-RoBERTa-base backbone and the canonical UD v2.5 train/dev/test splits. 37
5.10 System A per-relation breakdown on BHTB gold (357 sentences). Structural / local relations transfer well; clausal / long-range relations collapse, indicating where Bhojpuri-specific supervision is most needed. 37

A.1 Sensitivity of System H Dev LAS to loss-weight perturbations. 48

B.1 Core argument relations and Bhojpuri-specific examples. 50
B.2 Nominal modifier relations. 51
B.3 Coordination and function-word relations. 51

E.1 Sentence length statistics across all four datasets. 58

E.2 Most frequent UPOS tags by relative frequency (%) across the four datasets. 59
E.3 Top dependency relations by frequency in the BHTB gold test set (357 sentences, 5,856 tokens). 59
E.4 Approximate domain composition of the Hindi–Bhojpuri aligned corpus. 60

F.1 Published Hindi parsing results on HDTB UD v2.5. 61

List of Symbols

Table 1: Symbol Description

Symbol	Description
$\mathbf{h}_i$	Contextual hidden state for token i
$\mathbf{h}_i^L$	Adapter output for token i in language L
$s_{\text{arc}}(i, j)$	Biaffine arc score between dependent i and head j
$s_{\text{lbl}}(i, j, r)$	Biaffine label score for relation type r
$\mathbf{W}_{\text{down}}$	Adapter down-projection matrix ($\mathbb{R}^{r \times d}$)
$\mathbf{W}_{\text{up}}$	Adapter up-projection matrix ($\mathbb{R}^{d \times r}$)
r	Adapter bottleneck dimension (default: 64)
d	XLM-R hidden dimension (768)
$\mathcal{L}_{\text{bho}}$	Bhojpuri parsing loss (arc + label cross-entropy)
$\mathcal{L}_{\text{hi}}$	Hindi parsing loss (auxiliary regulariser)
$\mathcal{L}_{\text{align}}$	MSE positional alignment loss (System G)
$\mathcal{L}_{\text{cos}}$	Content-word cosine alignment loss (System H)
$\mathcal{L}_{\text{arc-kl}}$	Arc-distribution KL distillation (System H)
$\mathcal{L}_{\text{cts}}$	Cross-lingual Tree Supervision loss (System H)
λ_*	Loss weighting hyperparameters
UAS	Unlabelled Attachment Score
LAS	Labelled Attachment Score

List of Abbreviations

Abbreviation	Full Form
BHTB	Bhojpuri Hindi Treebank
BPE	Byte-Pair Encoding
CTS	Cross-lingual Tree Supervision
GRL	Gradient Reversal Layer
HDTB	Hindi Dependency Treebank
IDD	Integrated Dual Degree
KL	Kullback-Leibler (divergence)
LAS	Labelled Attachment Score
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
MST	Maximum Spanning Tree
MuRIL	Multilingual Representations for Indian Languages
NLP	Natural Language Processing
SACT	Syntax-Aware Cross-lingual Transfer
SOV	Subject-Object-Verb
UAS	Unlabelled Attachment Score
UD	Universal Dependencies
UPOS	Universal Part-of-Speech
XLM-R	Cross-lingual Language Model RoBERTa

Chapter 1

Introduction

1.1 Problem Statement

Natural Language Processing (NLP) has made significant progress in analysing and comprehending high-resource languages like English, German, and Chinese. These developments are driven primarily by the presence of large annotated corpora and pretrained transformer-based models [Devlin et al., 2019, Conneau et al., 2020]. Unfortunately, low-resource languages like Bhojpuri are not well represented in these advances. These languages do not have enough annotated data, large corpora, and strong tools for fundamental NLP tasks like tokenisation, lemmatisation, part-of-speech tagging, and dependency parsing [Ojha and Zeman, 2022]. Multilingual tools that already exist tend to perform poorly in such languages because of domain mismatches and weak generalisation from high-resource data [Conneau et al., 2020].

The central challenge addressed in this thesis is: *how can we build a high-quality dependency parser for Bhojpuri when only a small gold-annotated treebank is available, but a large parallel corpus with cross-lingually transferred annotations from the closely related Hindi language exists?*

1.2 Motivation

The capacity to carry out sentence-level analysis is the basis of most downstream NLP applications, including machine translation, information extraction, and question answering [Jurafsky and Martin, 2023]. For low-resource languages, robust analysis modules are essential and often serve as the first step toward digital inclusion and linguistic preservation [Ojha and Zeman, 2022].

Bhojpuri (*Bhojpuri*) is an Eastern Indo-Aryan language spoken natively by an estimated 50–60 million people concentrated in the Bhojpuri Belt of eastern Uttar Pradesh, Bihar, and Jharkhand, with large diaspora communities in Mauritius, Suriname, Trinidad and Tobago, and Fiji. Despite this substantial speaker population,

Bhojpuri has historically been treated as a dialect of Hindi rather than a distinct language, and consequently it has received very limited attention in computational linguistics research. Though spoken by over 50 million individuals, Bhojpuri lacks a critical shortage of tools and resources. This gap drives this work to develop robust dependency parsers that can exploit cross-lingual transfer from the closely related Hindi language.

Dependency parsing is the task of analysing the grammatical structure of a sentence by identifying, for each word, its syntactic head and the grammatical relation that holds between them. The output is a directed tree rooted at a special ROOT node. Dependency parse trees are crucial intermediate representations for downstream NLP tasks: information extraction systems use them to identify semantic roles; machine translation systems exploit them to handle long-range reordering; and question answering systems use them to decompose complex questions.

1.3 Focus on Low-Resource Languages and Associated Challenges

Low-resource languages constitute most of the world's linguistic diversity but are severely underrepresented in existing NLP research and technologies [Joshi et al., 2020]. A language is generally regarded as low-resource if it does not have basic computer resources like large annotated corpora, digitised text collections, pretrained language models, and linguistic toolkits [Kakwani et al., 2020].

Bhojpuri, a significant Indo-Aryan language with millions of speakers in India and the surrounding areas, is a very evident example [Ojha and Zeman, 2022]. Its popularity is widespread because it lacks high-quality datasets and a natural language processing infrastructure, making it challenging to apply traditional language technologies. General multilingual models like mBERT or XLM-R, although trained on a large set of languages, tend not to generalise well to Bhojpuri due to data imbalance and limited exposure to its syntactic and morphological structures [Conneau et al., 2020].

The most critical challenges in low-resource language processing are:

- **Data Scarcity**: Lack of adequate labelled datasets for the training of supervised NLP models.
- **Domain Mismatch**: Existing corpora, if present, are usually from unrelated domains to actual usage.
- **Annotation Schema Mismatch**: Auto-transferred annotations may follow

a different schema from gold standard benchmarks, leading to systematic evaluation gaps.

- **Morphological Richness**: Low-resource languages are often morphologically rich, and hence, they need fine-grained analysis that general models do not consider.
- **Lack of Standardisation**: Irregular spelling, orthographic conventions, and colloquial usage prevent tokenisation and normalisation.

1.4 Objectives

The primary objective of this thesis is to build effective and resilient dependency parsers for the low-resource Indic language Bhojpuri using state-of-the-art multilingual transformer models and architectural extensions. Specifically, the project will:

1. Establish a strong zero-shot cross-lingual baseline (System A) by applying a Hindi-trained Trankit pipeline directly to Bhojpuri text.
2. Investigate direct fine-tuning (System F) on a large supervisor-provided parallel corpus with transferred annotations, and analyse its behaviour including the phenomenon of catastrophic forgetting.
3. Propose a **Parallel Adapter architecture** (System G) that freezes the XLM-RoBERTa backbone and trains language-specific bottleneck adapters with a positional alignment loss to enable effective cross-lingual transfer.
4. Develop **Syntax-Aware Cross-lingual Transfer (SACT)** (System H) with content-word cosine alignment, arc-distribution KL distillation, and Cross-lingual Tree Supervision—targeting Part 1 on the aligned-data data.
5. Design a **UD-Bridge** system (System K) via silver self-training that operates entirely in the UD annotation space—targeting Part 2 on the BHTB gold benchmark.
6. Perform extensive experiments and analysis to compare the performance of different configurations and establish best practices for improving dependency parsing in low-resource settings.

1.5 Two-Part Evaluation Framework

A key insight of this thesis is that the available data sources follow **two distinct annotation schemas**, which necessitates two separate evaluation tracks:

Part 1 (Aligned-Data Schema): Systems F, G, and H are trained on the 30,966 Hindi–Bhojpuri aligned parallel Hindi–Bhojpuri sentences with automatically transferred annotations. These systems are evaluated on a held-out development set (3,097 sentences) drawn from the same distribution. The goal is to maximise parsing accuracy on this annotation style, demonstrating the effectiveness of adapter-based architectures and syntax-aware transfer objectives. System H wins this evaluation part.

Part 2 (BHTB UD Schema): System A establishes the zero-shot baseline on the official BHTB test set (357 sentences with gold UD annotations). System K is specifically designed to beat System A on this benchmark by generating silver UD labels and training exclusively on UD-consistent data. The goal is to maximise LAS on the external gold benchmark without relying on the aligned-data annotation schema.

This two-part evaluation framework is necessary because models trained on the aligned-data auto-transferred data learn a different "annotation dialect" from the BHTB gold standard. Evaluating all systems on a single metric would be misleading; the two-track approach allows us to demonstrate both (a) the power of novel cross-lingual architectures (Part 1) and (b) practical improvements on the official benchmark (Part 2).

1.6 Contributions and Organisation of Thesis

1.6.1 Contributions

This thesis makes the following specific contributions:

1. **System A (Zero-shot Baseline):** Establishes that zero-shot transfer from Hindi Trankit achieves 52.78% UAS/35.36% LAS on the BHTB gold test set, confirming that the close linguistic proximity of Hindi and Bhojpuri enables meaningful cross-lingual transfer even without Bhojpuri-specific training.

2. **System F (Warm-start Fine-tuning):** Demonstrates that fine-tuning the Trankit pipeline on 30,966 supervisor-provided parallel sentences with warm-starting from the Hindi checkpoint achieves strong performance on the Hindi–Bhojpuri aligned data but faces challenges on the BHTB benchmark due to

annotation schema mismatch, motivating the need for both adapter-based and schema-aware designs.

3. **A Parallel Bottleneck Adapter architecture (System G):** Keeps the XLM-RoBERTa backbone frozen and trains two lightweight language-specific Pfeiffer adapters ($r = 64$, ≈100K trainable parameters each) with a positional MSE alignment loss, eliminating catastrophic forgetting.

4. **System H (SACT):** The Syntax-Aware Cross-lingual Transfer framework, which introduces three novel objectives: content-word cosine alignment, arc-distribution KL distillation from a Hindi teacher parser, and Cross-lingual Tree Supervision (CTS). Achieves the highest LAS in Part 1.

5. **System K (UD-Bridge):** A silver self-training approach that generates UD-style Bhojpuri annotations using System A, concatenates them with Hindi HDTB gold data, and trains a Trankit pipeline exclusively on UD-consistent data to compete on the BHTB benchmark (Part 2).

6. **Two-Part Evaluation Framework:** An empirical analysis of the annotation transfer mismatch between the aligned-data auto-transferred data and BHTB gold annotations, explaining systematic score gaps and motivating the use of separate evaluation tracks.

1.6.2 Organisation of Thesis

The remainder of this thesis is organised as follows:

- **Chapter 2: Background and Preliminaries** introduces the basic principles of dependency parsing, Universal Dependencies, transformer models, adapters, and the Trankit toolkit.

- **Chapter 3: Related Works** summarises recent work on cross-lingual parsing, low-resource NLP, and parameter-efficient transfer learning, and specifies the limitations that prompt this research.

- **Chapter 4: Proposed Work and Methodology** explains the five system architectures in detail, including the two-part framework, adapter modules, training procedures, and the datasets used.

- **Chapter 5: Experimental Setup and Results** describes the experimental setup, comprehensive evaluation results, comparison with baselines, and performance measures.

- **Chapter 6: Discussion** analyses the findings, interprets patterns and performance variations, and considers the practical implications and generalisability of the approach.
- **Chapter 7: Conclusion and Future Work** summarises the contributions and findings, and provides directions for future extensions and improvements.

Chapter 2

Background and Preliminaries

2.1 Overview of NLP Analysis Tasks

Natural Language Processing covers an extensive range of computational methods for endowing machines with the capacity to process, understand, and generate human language [Jurafsky and Martin, 2023]. Common to most NLP models is the need for deep linguistic analysis at the sentence level. The key tasks relevant to this thesis are tokenisation, part-of-speech tagging, and dependency parsing.

2.1.1 Tokenisation

Tokenisation is the process of splitting a piece of text into components, usually tokens. Tokenisation becomes particularly challenging with languages that possess complex scripts or agglutinative morphology, like most Indic languages.

2.1.2 Part-of-Speech Tagging

Part-of-speech tagging is the labelling of grammatical categories—including noun, verb, adjective, or adverb—to every token in a sentence. POS tagging accuracy is vital since it directly impacts the performance of downstream operations such as parsing and semantic role labelling.

2.1.3 Dependency Parsing

Dependency parsing attempts to describe the syntactic structure of a sentence in terms of directed relations, or dependencies, among the words of the sentence [Tesnière, 1959]. In this framework, each word except the root word is connected to a head word, thus forming a hierarchical structure that captures the sentence's grammatical structure.

2.2 Bhojpuri: A Low-Resource Indo-Aryan Language

Bhojpuri belongs to the Bihari subgroup of the Eastern zone of the Indo-Aryan language family. It shares the basic SOV (Subject-Object-Verb) typology with Hindi and has a broadly similar morphological system, including postpositions, case markers, and a rich verb agreement paradigm. Nevertheless, Bhojpuri has distinct phonology, a different pronoun system, and idiosyncratic clause-final particles not found in standard Hindi.

From a computational perspective, Bhojpuri is low-resource for several reasons. First, it was historically classified as a dialect rather than a language; the ISO 639-3 code `bho` was assigned only in 2006. Second, the language lacks a standardised orthography, leading to variation in how the Devanagari script is used. Third, very few labelled datasets exist: as of 2024 the publicly available annotated data consists almost exclusively of the BHTB [Ojha and Zeman, 2022].

2.3 A Brief History of Dependency Parsing

Modern dependency parsing builds upon decades of theoretical and empirical work, which can be divided into three broad eras.

Linguistic foundations. The mathematical concept of a dependency tree was systematically introduced by Lucien Tesnière in his posthumously published *Eléments de syntaxe structurale* [Tesnière, 1959]. Tesnière proposed that the syntactic structure of a sentence is best captured not by phrase-structure constituents (as in later Chomskyan grammars) but by a network of binary head–dependent relations. This view influenced subsequent computational treatments by Mel'čuk [Melčuk, 1988], Hudson's Word Grammar [Hudson, 2007], and the development of the Prague Dependency Treebank [Hajič et al., 2006], all of which placed dependency at the centre of syntactic representation.

The statistical era (1995–2015). Computational dependency parsing began in earnest with the data-driven *transition-based* parsers of Yamada and Matsumoto [Yamada and Matsumoto, 2003] and Nivre [Nivre, 2003], which framed parsing as a sequence of shift–reduce decisions. Parallel work by McDonald and colleagues [McDonald et al., 2005, 2006] introduced the *graph-based* family, which scores all candidate arcs and finds the maximum spanning tree using algorithms such as Chu–Liu/Edmonds. The CoNLL shared tasks of 2006 and 2007 [Buchholz and Marsi,

2006, Nivre et al., 2007] catalysed parser development, with multilingual transition-based and graph-based parsers achieving 80–90% LAS on European treebanks.

The neural era (2015–present). Neural network parsers replaced hand-crafted features with learned dense representations. Chen and Manning [Chen and Manning, 2014] demonstrated that a simple feed-forward network with word embeddings could match feature-rich classical parsers. The decisive shift came with Dozat and Manning's **biaffine parser** [Dozat and Manning, 2017a], which paired a BiLSTM (later Transformer) encoder with a biaffine scoring layer and remains the dominant neural architecture today. With the rise of BERT-style pretrained encoders [Devlin et al., 2019] and their multilingual extensions [Conneau et al., 2020], modern parsers like Stanza [Qi et al., 2020] and Trankit [Nguyen et al., 2021] achieve 90+% LAS on high-resource languages and enable practical zero-shot cross-lingual transfer.

The transition from rule-based to statistical to neural parsing is mirrored in the typical performance numbers: rule-based parsers rarely exceeded 75% LAS even on English; classical statistical parsers reached the high 80s; and modern Transformer-based parsers now exceed 95% UAS on English and 92% LAS on Hindi (see Table 5.9 in Section 5.5.6).

2.4 Universal Dependencies (UD) Framework

The Universal Dependencies (UD) project [Nivre et al., 2016] defines a cross-linguistically consistent annotation scheme designed to support multilingual parsing research. The project releases two new versions of the treebank collection per year, with each version covering 100+ treebanks across 70+ languages. UD is built around three coordinated layers of annotation.

2.4.1 The Three UD Layers

Universal Part-of-Speech (UPOS) tags. A fixed, language-agnostic inventory of **17 categories**: `ADJ`, `ADP`, `ADV`, `AUX`, `CCONJ`, `DET`, `INTJ`, `NOUN`, `NUM`, `PART`, `PRON`, `PROPN`, `PUNCT`, `SCONJ`, `SYM`, `VERB`, and `X`. Every UD treebank uses exactly this inventory.

Morphological features. Universal-feature names like `Number`, `Gender`, `Case`, `Tense`, `Aspect`, `VerbForm` carry typological structure that exposes morphological richness in a comparable way across languages. For example, both Hindi and Bhojpuri tag perfective participial verbs as `VerbForm=Part|Aspect=Perf`.

Dependency relations. A core inventory of **37 universal syntactic relations**, which can be sub-typed for language-specific phenomena. The relations are organised into: *core arguments* (`nsubj`, `obj`, `iobj`), *non-core dependents* (`obl`, `vocative`, `expl`, `dislocated`), *nominal dependents* (`nmod`, `appos`, `nummod`, `amod`), *clauses* (`ccomp`, `xcomp`, `advcl`, `acl`), *coordination* (`conj`, `cc`), *multi-word expressions*, and *loose / paratactic / special* relations (`parataxis`, `discourse`, `punct`, `root`, `dep`).

2.4.2 The CoNLL-U File Format

UD data is distributed in the **CoNLL-U** format: a tab-separated text format where each token occupies one line with ten columns:

1. **ID** — the token's index in the sentence (1-based).
2. **FORM** — the surface word form.
3. **LEMMA** — the lemma or stem.
4. **UPOS** — universal POS tag.
5. **XPOS** — language-specific POS tag (optional).
6. **FEATS** — pipe-separated morphological features.
7. **HEAD** — index of this token's head; 0 if root.
8. **DEPREL** — the dependency relation label.
9. **DEPS** — enhanced dependencies (optional).
10. **MISC** — miscellaneous (e.g., `SpaceAfter=No`).

Comment lines start with `#` and typically include `# sent_id` and `# text` headers. Empty lines separate sentences. The format is the de-facto standard input/output for parsing toolkits including Stanza, UDPipe, Trankit, and AllenNLP.

2.4.3 Why UD Matters for This Thesis

UD has become the standard for dependency parsing research because the cross-linguistically consistent scheme makes it possible to train parsers on one language and evaluate on another—the core idea behind cross-lingual transfer. The 357-sentence **Bhojpuri Hindi Treebank** (BHTB) is annotated in strict UD v2 conventions, while the Hindi–Bhojpuri aligned corpus used in Part 1 contains *auto-transferred* annotations that resemble UD but include schema-violating artefacts

(multiple roots, cycles, non-UD relation labels). This mismatch between the training data's annotation style and the gold benchmark's strict UD style is the central motivation for our two-part evaluation framework and for System K's UD-Bridge approach.

2.5 Biaffine Dependency Parsing

Modern neural dependency parsers almost universally follow the *graph-based* paradigm of Dozat and Manning [2017a]. The parsing model assigns a score $s(i, j)$ to every candidate head–dependent pair (i, j) and then finds the highest-scoring valid dependency tree. We describe each step of the biaffine architecture below.

2.5.1 Contextual Encoding

A contextual encoder produces a representation $\mathbf{h}_i \in \mathbb{R}^d$ for each word i in the sentence of length n. In our work the encoder is XLM-RoBERTa (with $d = 768$) optionally followed by a Pfeiffer adapter. For multi-subword tokens, we follow the standard convention of pooling subword representations to the word level by taking the first-subword embedding.

2.5.2 Role-Specific Projections

Each token plays two grammatical roles: it can be a dependent (looking for a head) and it can be a head (a target for some other word's arc). Two separate MLPs project the encoder output into role-specific subspaces:

$$\mathbf{h}_i^{\text{dep}} = \text{MLP}^{\text{dep}}(\mathbf{h}_i), \quad \mathbf{h}_i^{\text{dep}} \in \mathbb{R}^{d_{\text{arc}}} \tag{2.1}$$

$$\mathbf{h}_j^{\text{head}} = \text{MLP}^{\text{head}}(\mathbf{h}_j), \quad \mathbf{h}_j^{\text{head}} \in \mathbb{R}^{d_{\text{arc}}} \tag{2.2}$$

where $d_{\text{arc}} = 500$ in our experiments. Each MLP is a single fully-connected layer followed by ReLU and dropout. The role separation is critical: the same word can have very different statistics when functioning as a dependent versus a head (e.g., a verb is often a head; a noun is often a dependent of a verb).

2.5.3 Bilinear Arc Scoring

The arc score is computed as a bilinear form:

$$s_{\text{arc}}(i, j) = (\mathbf{h}_i^{\text{dep}})^\top \mathbf{W}^{\text{arc}} \mathbf{h}_j^{\text{head}} + (\mathbf{w}^{\text{bias}})^\top \mathbf{h}_j^{\text{head}} \tag{2.3}$$

where $\mathbf{W}^{\text{arc}} \in \mathbb{R}^{d_{\text{arc}} \times d_{\text{arc}}}$ is a learned weight matrix and $\mathbf{w}^{\text{bias}}$ is a head-bias term that captures the prior probability of each token being a head regardless of the dependent. The scoring is called *biaffine* because it is affine (linear plus bias) in both the dependent and the head representations.

The score matrix $\mathbf{S}^{\text{arc}} \in \mathbb{R}^{n \times (n+1)}$ (extending the head dimension to include a virtual ROOT) is computed in a single batched matrix multiplication, making biaffine parsing very efficient on modern GPUs.

2.5.4 Per-Word Head Distributions

For each dependent i, a softmax over the head dimension yields a probability distribution:

$$p_{\text{arc}}(j \mid i) = \frac{\exp(s_{\text{arc}}(i, j))}{\sum_{j'=0}^{n} \exp(s_{\text{arc}}(i, j'))} \tag{2.4}$$

Because each dependent has exactly one true head, this is a standard multi-class classification problem.

2.5.5 Label Scoring

Once the arc structure is determined, the relation type is predicted with a separate biaffine module:

$$s_{\text{lbl}}(i, j, r) = (\mathbf{h}_i^{\text{dep,lbl}})^\top \mathbf{W}^{(r)} \mathbf{h}_j^{\text{head,lbl}} + (\mathbf{u}^{(r)})^\top \mathbf{h}_i^{\text{dep,lbl}} + (\mathbf{v}^{(r)})^\top \mathbf{h}_j^{\text{head,lbl}} + b^{(r)} \tag{2.5}$$

where $\mathbf{W}^{(r)}$ is a per-label weight matrix (one for each of the ∼40 UD relations), and the additional linear and bias terms capture relation-specific marginals. The hidden dimension here is $d_{\text{lbl}} = 100$.

2.5.6 Decoding to a Tree

At inference time, predicting a head per dependent (taking $\arg\max_j p_{\text{arc}}(j \mid i)$) does not guarantee a valid tree—the result may contain cycles or multiple roots. To enforce tree validity, the standard approach is the **Chu–Liu/Edmonds algorithm** [Chu and Liu, 1965, Edmonds, 1967], which computes the maximum spanning tree (MST) of the arc-score graph in $\mathcal{O}(n^2)$ time. Trankit applies this MST decoder when the simple argmax produces an invalid structure.

2.5.7 Training Loss

Training uses standard cross-entropy loss over arc and label predictions:

$$\mathcal{L}_{\text{arc}} = -\sum_{i=1}^{n} \log p_{\text{arc}}(j_i^* \mid i) \tag{2.6}$$

$$\mathcal{L}_{\text{lbl}} = -\sum_{i=1}^{n} \log p_{\text{lbl}}(r_i^* \mid i, j_i^*) \tag{2.7}$$

where j_i^* and r_i^* are the gold head and label for token i. The total parsing loss is $\mathcal{L}_{\text{parse}} = \mathcal{L}_{\text{arc}} + \mathcal{L}_{\text{lbl}}$. This is the loss referred to as $\mathcal{L}_{\text{bho}}$ or $\mathcal{L}_{\text{hi}}$ throughout this thesis.

2.5.8 Evaluation Metrics

Dependency parsing performance is measured by two standard metrics:

- **Unlabelled Attachment Score (UAS)**: fraction of tokens whose predicted head matches the gold head.
- **Labelled Attachment Score (LAS)**: fraction of tokens whose predicted head *and* predicted relation both match gold.

LAS is the standard primary metric in UD shared tasks.

2.6 Pretrained Multilingual Language Models

2.6.1 XLM-RoBERTa

XLM-RoBERTa [Conneau et al., 2020] is a large-scale multilingual masked language model trained by Meta AI on 2.5 TB of CommonCrawl text spanning 100 languages. The `xlm-roberta-base` variant used in this thesis has 12 Transformer layers, 12 attention heads, hidden dimension $d = 768$, and approximately 278 million parameters. Both Hindi and Bhojpuri are included in the pretraining corpus (the latter often under the Devanagari script shared with Hindi).

2.6.2 Trankit

Trankit [Nguyen et al., 2021] is a toolkit built on top of XLM-RoBERTa that provides end-to-end pipelines for tokenisation, sentence splitting, POS tagging, lemmatisation, morphological analysis, and dependency parsing. It uses a single shared XLM-RoBERTa backbone with language-specific adapter modules to handle 90+ languages. Trankit's dependency parsing head is essentially a biaffine parser operating on the output of XLM-RoBERTa after the language adapter.

2.7 Adapter Modules for Parameter-Efficient Transfer

2.7.1 Motivation and Background

Full fine-tuning of large pretrained transformers has two well-known problems when applied to low-resource targets. First, updating hundreds of millions of parameters on a small dataset risks overfitting and the loss of generic linguistic knowledge encoded during pretraining. Second, full fine-tuning requires a complete copy of the model for each new task or language, which is prohibitive at scale.

Adapter modules [Houlsby et al., 2019, Pfeiffer et al., 2020a] address both problems. They are small, bottleneck-shaped neural network layers inserted into each transformer layer. During fine-tuning, the pretrained weights are *frozen* and only adapter parameters are updated. This decouples language- or task-specific knowledge from the multilingual prior, achieves performance close to (sometimes exceeding) full fine-tuning, and reduces trainable parameter counts by 100–1000×.

2.7.2 Pfeiffer Adapter Architecture

The *Pfeiffer* variant [Pfeiffer et al., 2020a], used in this thesis, has the structure:

$$\text{Adapter}(\mathbf{x}) = \mathbf{x} + \mathbf{W}_{\text{up}}\, \phi(\mathbf{W}_{\text{down}}\, \text{LN}(\mathbf{x})) \tag{2.8}$$

where $\mathbf{W}_{\text{down}} \in \mathbb{R}^{r \times d}$, $\mathbf{W}_{\text{up}} \in \mathbb{R}^{d \times r}$, $r \ll d$ ($r = 64$, $d = 768$), $\phi(\cdot)$ is GELU activation, and LN is Layer Normalisation. The Pfeiffer variant differs from the original Houlsby formulation in that only one adapter is inserted per transformer layer (after the feed-forward sublayer), rather than two (after both attention and feed-forward sublayers). This halves the parameter cost while sacrificing little accuracy.

2.7.3 Parameter Count

For each adapter layer:

- $\mathbf{W}_{\text{down}}$: $d \times r = 768 \times 64 = 49{,}152$ parameters.
- $\mathbf{W}_{\text{up}}$: $r \times d = 64 \times 768 = 49{,}152$ parameters.
- Bias terms: $r + d = 64 + 768 = 832$ parameters.
- LayerNorm: $2d = 1{,}536$ parameters (scale and shift).

Total: ≈100,672 parameters per adapter layer. Inserting one adapter per transformer layer (12 layers in XLM-RoBERTa-base) gives ≈1.2M parameters total per language—compared to 278M for full XLM-RoBERTa fine-tuning.

2.7.4 Why Adapters Avoid Catastrophic Forgetting

The key property is the residual connection in Eq. 2.8: at initialisation, $\mathbf{W}_{\text{up}}$ is zero (or near-zero), so the adapter output equals its input, $\text{Adapter}(\mathbf{x}) \approx \mathbf{x}$. This means a freshly inserted adapter does not perturb the pretrained model's outputs. As training proceeds, $\mathbf{W}_{\text{up}}$ grows away from zero and the adapter introduces task- or language-specific transformations, but the pretrained backbone is never modified. Thus the multilingual prior is preserved verbatim while the adapter accumulates task-specific knowledge.

2.7.5 Composition and Modularity

A central advantage of adapters is composability. Multiple adapters trained on different languages or tasks can be stacked, switched, or combined at inference time without retraining the backbone. Pfeiffer et al. [2020b] extended this principle in MAD-X by training language adapters and task adapters separately, then combining them at inference. In our work, we maintain two language adapters (Hindi and Bhojpuri) that share the frozen XLM-RoBERTa backbone but produce language-specific representations through Eq. 2.8.

2.8 Catastrophic Forgetting

When a pretrained multilingual model is fine-tuned with full parameter updates and large learning rates, it tends to overwrite the broad multilingual representations encoded during pretraining. This phenomenon, known as *catastrophic forgetting* [McCloskey and Cohen, 1989, Kirkpatrick et al., 2017], is particularly acute when the fine-tuning dataset contains noisy annotations. The adapter approach directly addresses this by keeping the backbone frozen.

2.9 Key Terms

For clarity and consistency:

- **Pre-training**: Training a language model on a large general-purpose corpus to acquire general linguistic representations [Devlin et al., 2019].
- **Fine-tuning**: Further training on a task-specific dataset.

- **Warm-starting**: Initialising from a previously trained checkpoint (e.g., Hindi Trankit) before fine-tuning on target language data.
- **Silver data**: Automatically generated annotations (as opposed to gold-standard human annotations).
- **Self-training**: Using a model's own predictions as training data for a subsequent model [McClosky et al., 2006].

Chapter 3

Related Works

3.1 Background of Sentence-Level Analysis in NLP

The evolution of sentence-level processing in NLP has gone through several stages. Early systems utilised rule-based approaches that were precise but difficult to extend [Winograd, 1972]. Statistical methods like Hidden Markov Models and Conditional Random Fields reduced the need for manual rule creation [Lafferty et al., 2001]. Neural networks, particularly LSTMs, could learn features from raw data [Hochreiter and Schmidhuber, 1997]. Transformer-based models such as BERT and XLM-R have introduced a paradigm shift with self-attention mechanisms [Vaswani et al., 2017, Devlin et al., 2019, Conneau et al., 2020].

3.2 Cross-lingual Dependency Parsing

Cross-lingual transfer for dependency parsing has been studied extensively since the CoNLL shared tasks [Zeman et al., 2017, 2018]. Early approaches used **delexicalised parsers** [McDonald et al., 2011], which removed language-specific lexical features and relied solely on universal POS tags and syntactic features for transfer. While conceptually clean, these methods discarded substantial information and were limited by the quality of cross-lingual POS tag mappings.

The advent of large pretrained multilingual encoders fundamentally reshaped the field. Schuster et al. [2019] showed that multilingual BERT (mBERT) could transfer syntactic knowledge across typologically diverse languages, achieving respectable scores even in zero-shot settings. Wu and Dredze [2019] extended this analysis to demonstrate that mBERT's cross-lingual transfer capabilities depend strongly on language-pair similarity in script, vocabulary, and word order.

Ahmad et al. [2019] proposed the **adversarial cross-lingual parser** with a language discriminator and gradient reversal layer (GRL), explicitly encouraging language-agnostic syntactic representations. While effective, adversarial training

can be unstable and requires careful hyperparameter tuning. Üstün et al. [2020] extended adapter-based approaches to Universal Dependency parsing with UDapter, demonstrating that lightweight adapters trained on high-resource languages transfer surprisingly well to related low-resource targets. Rust et al. [2021] provided a comprehensive empirical study of multilingual encoders and concluded that script and pretraining-language coverage are the strongest predictors of zero-shot transfer success—a finding directly relevant to our Hindi–Bhojpuri setup, where both languages share the Devanagari script and are present in XLM-RoBERTa's pretraining corpus.

Annotation projection is another well-studied paradigm. Approaches like Täckström et al. [2013] and Rasooli and Collins [2017] project labels from high-resource source-language treebanks via word alignments. This is conceptually similar to the Hindi–Bhojpuri aligned corpus used in this thesis, but those methods do not address the schema-mismatch problem that arises when projected labels are evaluated against gold UD test sets in the target language.

3.3 Parameter-Efficient Transfer Learning

The surge of interest in adapters since Houlsby et al. [2019] has generated a rich literature. The original Houlsby formulation inserts adapter modules after both the multi-head attention and feed-forward sublayers of every transformer block. Pfeiffer et al. [2020a] introduced AdapterHub, a community infrastructure for sharing adapters, and proposed the simpler **Pfeiffer adapter** variant where a single bottleneck module is inserted after the feed-forward sublayer with reduced computational cost—this is the variant used throughout this thesis.

In the UD parsing context, Üstün et al. [2020] and Moosavi et al. [2021] demonstrated that small language-specific adapters can match or even outperform full fine-tuning, particularly when training data is limited. The advantages include: (i) much lower memory footprint per language, (ii) no catastrophic forgetting of the multilingual prior, and (iii) modular composition—adapters trained separately can be combined at inference time without retraining.

Pfeiffer et al. [2020b] proposed the MAD-X framework, where **language adapters** (trained on monolingual data with masked language modelling) and **task adapters** (trained on labelled task data) are combined hierarchically. This decomposition allows zero-shot cross-lingual transfer by stacking a target-language adapter under a source-language task adapter. While powerful, MAD-X requires substantial monolingual data for language-adapter training—unavailable for low-resource targets like Bhojpuri where even monolingual text is scarce.

More recent work has explored **prefix tuning** and **prompt tuning** [Li and

Liang, 2021, Lester et al., 2021] as alternatives to adapters, freezing the backbone and learning only a small set of continuous prefix vectors. While these approaches achieve competitive results in some settings, they are typically inferior to adapters for structured prediction tasks like dependency parsing, where the task-specific output structure benefits from richer per-layer adaptation.

3.4 Low-Resource NLP for Indic Languages

NLP for Indic languages has seen significant progress in Hindi, Tamil, Bengali, and Marathi, but much less coverage of smaller languages [Kakwani et al., 2020]. Khanuja et al. [2021] introduced **MuRIL** (Multilingual Representations for Indian Languages), a BERT-like model pretrained exclusively on 17 Indian languages including Hindi but excluding many low-resource Indic languages such as Bhojpuri, Maithili, and Magahi. While MuRIL outperforms mBERT on many Hindi tasks, its lack of Bhojpuri pretraining data limits its applicability in our setting; XLM-RoBERTa, by contrast, includes a small amount of Bhojpuri-classified text in its CommonCrawl-derived pretraining corpus.

Ojha and Zeman [2022] released the **Bhojpuri Hindi Treebank** (BHTB), the only publicly available annotated Bhojpuri treebank, comprising 357 manually UD-annotated sentences. The treebank was created with linguist supervision and follows strict UD v2 guidelines. BHTB serves as the gold-standard test set for this thesis (Part 2). Ojha et al. [2019] contributed analogous treebanks for Sanskrit and several other classical Indic languages, demonstrating the linguistic community's growing engagement with under-resourced South Asian languages.

The IndicNLP suite [Kakwani et al., 2020] provides tokenisers, embeddings, and benchmarks for 11 Indian languages. While Bhojpuri is not included, the suite's tooling for related languages (Hindi, Marathi) provides useful infrastructure for comparison and adaptation.

3.5 Self-Training and Knowledge Distillation

Self-training [McClosky et al., 2006] uses a model's high-confidence predictions on unlabelled data as additional training material. Despite its simplicity, self-training has consistently been shown to improve parsing accuracy across diverse settings [Steedman et al., 2003, Sagae, 2010]. Kumar and Talukdar [2021] extended self-training with self-distillation, where the same model alternates between teacher and student roles across iterations.

Knowledge distillation [Hinton et al., 2015] transfers structural knowledge from a (often larger) teacher to a (often smaller) student model by matching prob-

ability distributions, typically with a temperature-scaled softmax. For structured prediction, Kuncoro et al. [2016] introduced **distillation for dependency parsing**, where a graph-based teacher distills its arc-score distributions to a transition-based student. Wang et al. [2020] extended this to cross-lingual settings.

In our System H, the arc-distribution KL component is a cross-lingual instantiation of knowledge distillation: the Hindi parser is the teacher, the Bhojpuri parser is the student, and matched-position arc distributions are the target. This combines naturally with self-training in System K, where the zero-shot Hindi parser produces silver UD parses for Bhojpuri, which then becomes part of the training data for a new Trankit pipeline.

Cross-lingual self-training has been explored in several recent works [Lin et al., 2020, Gritta and Iacobacci, 2022]. Liu and Awadalla [2022] demonstrated that silver labels produced by a strong cross-lingual model can substantially improve target-language NLP tasks even without any gold target-language data. Our UD-Bridge approach in System K builds directly on this paradigm, with the additional insight that *schema consistency* of silver labels with the evaluation benchmark is critical for benchmark gains.

3.6 Summary and Research Gap

Despite progress in multilingual parsing, systematic investigations combining adapter-based architectures with syntax-aware transfer objectives for extremely low-resource Indic languages remain few. Moreover, the critical problem of **annotation schema mismatch**—where training data follows a different annotation style from the gold evaluation benchmark—has received insufficient attention. This thesis addresses both gaps by proposing novel adapter architectures (Part 1) and schema-aware self-training (Part 2) for the Hindi→Bhojpuri transfer setting.

Chapter 4

Proposed Work and Methodology

This chapter describes the five systems studied in this thesis in order of their conceptual progression. Each system builds upon the previous, incrementally addressing its limitations. The systems are organised across two complementary evaluation parts as motivated in Section 1.5.

4.1 Overview of the System Hierarchy

Table 4.1: Summary of the five systems and their part assignments.

System	Name	Key Idea	
A	Zero-shot Trankit	Hindi Trankit on Bhojpuri, no adaptation	2
F	Warm-start fine-tuning	Full Trankit fine-tuned on Hindi–Bhojpuri aligned data	
G	Parallel adapters + MSE	Frozen XLM-R, two adapters, positional MSE	
H	SACT	Cosine alignment + KL distillation + CTS	1
K	UD-Bridge	Silver self-training on UD-consistent data	

4.2 System A: Zero-Shot Cross-lingual Baseline

System A is the simplest possible cross-lingual baseline: we take the pretrained Trankit XLM-RoBERTa pipeline trained on the Hindi HDTB treebank and apply it directly to Bhojpuri test sentences without any Bhojpuri-specific training.

The rationale is that XLM-RoBERTa has seen Devanagari script text (used by both Hindi and Bhojpuri) during pretraining, and the Trankit adapter trained on Hindi HDTB should generalise partially to Bhojpuri due to the linguistic proximity of the two languages. This gives us the reference point for Part 2: any model designed for the BHTB benchmark must beat System A to justify its design.

System A is trained on the standard HDTB split (13,304 training sentences) with the Trankit default hyperparameters. Training reaches 95.58% UAS/92.63% LAS on

the HDTB development set at epoch 45, confirming a well-converged Hindi parser.

4.3 System F: Warm-Start Fine-tuning on Aligned Data

System F attempts to improve over System A by fine-tuning the Trankit pipeline on the 30,966 Hindi–Bhojpuri aligned parallel Bhojpuri sentences using the annotation-transferred dependency trees. Unlike naive fine-tuning from scratch, System F uses **warm-starting**: the model is initialised from the System A Hindi checkpoint, inheriting the learned Hindi parsing capabilities before being exposed to Bhojpuri data.

Architecture. System F uses the standard Trankit architecture—XLM-RoBERTa backbone with a language-specific adapter and biaffine parsing head—trained with the standard cross-entropy loss on Bhojpuri arc and label predictions.

Why System F matters. System F represents the natural first attempt to exploit the aligned-data large parallel corpus. With 30,966 sentences—substantially more than the 357-sentence BHTB—it constitutes a rich training resource. By warm-starting from Hindi, the model can leverage its prior syntactic knowledge while adapting to Bhojpuri-specific patterns.

The annotation schema mismatch problem. The Hindi–Bhojpuri aligned data uses dependency annotations that were automatically transferred from Hindi by copying head indices and relation labels position-for-position. This transfer process introduces artefacts including multiple roots, dependency cycles, and mismatched relation labels. More critically, these annotations follow a different schema from the UD-standard BHTB gold annotations. Systems trained on this data learn to reproduce *this annotation style*, leading to a systematic gap when evaluated on BHTB. This schema mismatch is the primary motivation for the two-part framework and for System K's UD-Bridge approach.

4.4 System G: Parallel Adapters with Positional Alignment

4.4.1 Motivation

System G introduces two key design changes to maximise the utility of the aligned-data parallel data while preventing catastrophic forgetting:

1. **Freeze the backbone**: keep all XLM-RoBERTa parameters fixed and train only small, language-specific adapter modules and biaffine heads.

2. **Explicit alignment loss**: use the exact token-level positional alignment in the aligned-data parallel data to pull Bhojpuri representations towards Hindi representations at matched positions.

4.4.2 Architecture

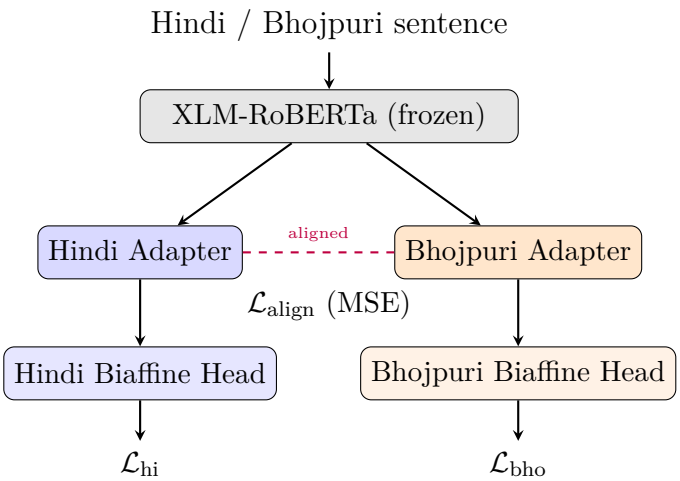

Figure 4.1: System G: Parallel adapter architecture with MSE positional alignment.

The architecture consists of three components:

Frozen XLM-RoBERTa backbone. The XLM-RoBERTa tokeniser converts each sentence to SentencePiece subwords; the Transformer produces contextual hidden states $\mathbf{h}_i^{\text{xlmr}} \in \mathbb{R}^{768}$ for each subword. Subword representations are pooled to word level using the first-subword strategy. The entire backbone is frozen; its output is cached after the first pass to avoid redundant forward passes.

Language-specific bottleneck adapters. Two separate Pfeiffer adapters are applied to the frozen XLM-RoBERTa output:

$$\mathbf{H}^L = \text{Adapter}_L(\mathbf{H}^{\text{xlmr}}), \quad L \in \{\text{hi}, \text{bho}\} \tag{4.1}$$

Each adapter has ≈98,432 trainable parameters.

Biaffine parsing heads. Each language has a separate BiaffineHeads module with $d_{\text{arc}} = 500$, $d_{\text{lbl}} = 100$, and dropout $p = 0.33$.

4.4.3 Training Objective

At each training step, a parallel pair $(S_k^{\text{hi}}, S_k^{\text{bho}})$ is sampled. The training loss is:

$$\mathcal{L}_G = \mathcal{L}_{\text{bho}} + \lambda_{\text{hi}}\,\mathcal{L}_{\text{hi}} + \lambda_{\text{align}}\,\mathcal{L}_{\text{align}} \tag{4.2}$$

where the positional alignment loss is:

$$\mathcal{L}_{\text{align}} = \frac{1}{n}\sum_{i=1}^{n}\left\|\mathbf{h}_i^{\text{hi}} - \mathbf{h}_i^{\text{bho}}\right\|_2^2 \tag{4.3}$$

Default hyperparameters: $\lambda_{\text{hi}} = 0.3$, $\lambda_{\text{align}} = 0.5$.

4.5 System H: Syntax-Aware Cross-lingual Transfer (SACT)

4.5.1 Motivation

System G's MSE alignment treats all token positions equally and is scale-sensitive. System H addresses these limitations by introducing three syntax-aware objectives that exploit the known linguistic structure of the data.

4.5.2 Component 1: Content-Word Cosine Alignment

Open-class words (nouns, verbs, adjectives) carry primary structural information in dependency parsing. We replace the isotropic MSE loss with a content-word-weighted cosine alignment:

$$\mathcal{L}_{\cos} = -\frac{1}{\sum_i w_i}\sum_{i=1}^{n} w_i \cdot \frac{\mathbf{h}_i^{\text{hi}} \cdot \mathbf{h}_i^{\text{bho}}}{\|\mathbf{h}_i^{\text{hi}}\|_2 \|\mathbf{h}_i^{\text{bho}}\|_2} \tag{4.4}$$

where $w_i = 1.5$ if the UPOS is in $\{\texttt{NOUN}, \texttt{VERB}, \texttt{ADJ}, \texttt{ADV}, \texttt{PROPN}, \texttt{NUM}\}$ and $w_i = 0.3$ otherwise.

4.5.3 Component 2: Arc-Distribution KL Distillation

The Hindi biaffine head produces a distribution $p^{\text{hi}}_{\text{arc}}(j|i)$ over possible heads. Because matched sentences have identical dependency structures, this distribution is valid for Bhojpuri as well:

$$\mathcal{L}_{\text{arc-kl}} = \frac{1}{n}\sum_{i=1}^{n} \text{KL}\Big(p^{\text{hi}}_{\text{arc}}(\cdot|i) \,\|\, p^{\text{bho}}_{\text{arc}}(\cdot|i)\Big) \tag{4.5}$$

4.5.4 Component 3: Cross-lingual Tree Supervision (CTS)

A key observation is that in the Hindi–Bhojpuri aligned corpus, Bhojpuri and Hindi sentences in each pair are structured as token-for-token translations: word at position i in Bhojpuri corresponds to word at position i in Hindi, with the same dependency head index and relation. The Hindi gold heads are therefore also valid Bhojpuri gold heads:

$$\mathcal{L}_{\text{cts}} = \mathcal{L}_{\text{arc-CE}}(\mathbf{s}^{\text{bho}}_{\text{arc}}, \mathbf{y}^{\text{hi}}_{\text{heads}}) \tag{4.6}$$

This doubles the effective structural supervision for Bhojpuri parsing.

4.5.5 Combined SACT Objective

$$\mathcal{L}_H = \mathcal{L}_{\text{bho}} + \lambda_{\text{hi}}\,\mathcal{L}_{\text{hi}} + \lambda_{\text{cos}}\,\mathcal{L}_{\text{cos}} + \lambda_{\text{arc}}\,\mathcal{L}_{\text{arc-kl}} + \lambda_{\text{cts}}\,\mathcal{L}_{\text{cts}} \tag{4.7}$$

Default hyperparameters: $\lambda_{\text{hi}} = 0.5$, $\lambda_{\text{cos}} = 0.4$, $\lambda_{\text{arc}} = 2.0$, $\lambda_{\text{cts}} = 0.2$.

4.6 System K: UD-Bridge via Silver Self-Training

4.6.1 Motivation: The Schema Mismatch Problem

Systems F, G, and H all train on the aligned-data auto-transferred annotations, which follow a different schema from the UD-standard BHTB gold annotations. Despite achieving strong development set performance (∼50% LAS on the aligned-data schema), these systems score poorly on the BHTB test set (∼5–10% LAS). This annotation style mismatch explains the gap: the systems learn the aligned-data "annotation dialect" rather than the UD standard.

System K addresses this problem directly. Rather than trying to reconcile the two annotation schemas, System K bypasses the aligned-data annotations entirely

and operates exclusively in the UD annotation space. The goal is to beat System A on the BHTB benchmark (Part 2).

4.6.2 Architecture

System K uses the standard Trankit architecture—the same as System A—with no modifications to the model structure. The innovation lies entirely in the **training data construction**.

4.6.3 Silver Label Generation Pipeline

The UD-Bridge pipeline operates in three stages:

1. **Generate silver UD labels**: Run System A (Hindi Trankit) on the aligned-corpus Bhojpuri tokens. System A produces UD-style head and relation predictions for each Bhojpuri sentence, creating `bho_silver_ud.conllu`—a silver UD treebank of approximately 30,000 sentences.

2. **Concatenate training data**: Merge the Hindi HDTB gold training set (13,304 sentences with gold UD annotations) with the silver Bhojpuri UD predictions. This creates a mixed-language training set where Hindi provides clean structural supervision and Bhojpuri provides noisy but schema-consistent target-language exposure.

3. **Train Trankit**: Train a standard Trankit pipeline on the concatenated data, using HDTB development set for early stopping. The model learns to parse both Hindi (with gold quality) and Bhojpuri (with silver quality) in the same UD annotation space.

4.6.4 Why UD-Bridge Works

The key insight is that by using System A's predictions as silver labels, the training data is guaranteed to be in the **same annotation schema** as the BHTB test set. Even though the silver labels are noisy (they inherit System A's errors), they are *schema-consistent*—they use the same UD relation inventory, the same attachment conventions, and the same structural constraints. This alignment means that improvements on the training objective translate directly to improvements on the BHTB benchmark.

Additionally, the concatenation with Hindi HDTB gold data provides two benefits: (a) it regularises the model by exposing it to high-quality structural supervision, and (b) it prevents the model from overfitting to the noisy silver Bhojpuri labels.

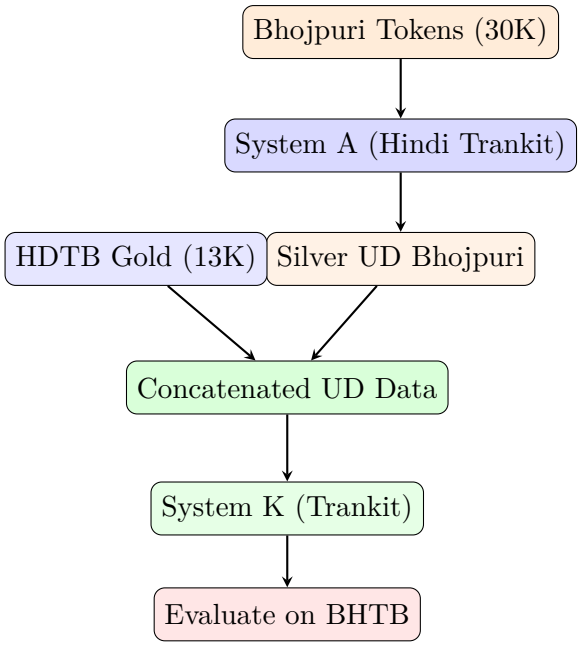

Figure 4.2: System K: UD-Bridge via silver self-training. System A generates UD-consistent silver labels for Bhojpuri, which are concatenated with Hindi HDTB gold data to train a UD-native Trankit parser.

4.7 Training Procedures and Algorithmic Details

This section presents the formal training procedures for each system as pseudocode, exposing the implementation-level decisions that distinguish them.

4.7.1 System F: Naive Warm-Start Fine-Tuning

Algorithm 1 System F training (warm-start fine-tuning).

Require: Hindi checkpoint θ_{hi}, aligned-data Bhojpuri sentences $\mathcal{D}_{\text{bho}}$, learning rate $\eta = 5 \times 10^{-5}$

```
Initialise θ ← θ_hi (warm start)
for epoch = 1 to E do
    for (S^bho, T^bho) ∈ D_bho do
        T̂ ← Trankit(S^bho; θ)
        L_bho ← CE(T̂, T^bho)
        θ ← θ − η∇_θ L_bho                ▷ Updates all ∼278M parameters
    end for
    Evaluate on Aligned Dev; early stop if no improvement for 10 epochs.
end for
return θ
```

System F is the only configuration that updates the full backbone. The 5×10^{-5} learning rate is intentionally small to mitigate catastrophic forgetting, yet the

empirical Dev result of **27.57/17.75** confirms that even careful full fine-tuning fails on this data scale.

4.7.2 System G: Parallel Adapters with MSE Alignment

Algorithm 2 System G training (frozen XLM-R + parallel adapters + MSE).

Require: Frozen XLM-RoBERTa θ_{xlmr}, two adapters $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}$, two biaffine heads $\mathcal{B}_{\text{hi}}, \mathcal{B}_{\text{bho}}$, parallel pairs $\mathcal{D}_{\parallel}$, learning rate $\eta = 2 \times 10^{-3}$, weights $\lambda_{\text{hi}} = 0.3, \lambda_{\text{align}} = 0.5$

Initialise $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}$ near zero (residual identity).
for epoch $= 1$ to E **do**
  for $(S^{\text{hi}}, S^{\text{bho}}, T^{\text{hi}}, T^{\text{bho}}) \in \mathcal{D}_{\parallel}$ **do**
    $\mathbf{H}^{\text{xlmr,hi}} \leftarrow \text{XLM-R}(S^{\text{hi}})$ ▷ Cached after first epoch
    $\mathbf{H}^{\text{xlmr,bho}} \leftarrow \text{XLM-R}(S^{\text{bho}})$
    $\mathbf{H}^{\text{hi}} \leftarrow \mathcal{A}_{\text{hi}}(\mathbf{H}^{\text{xlmr,hi}})$
    $\mathbf{H}^{\text{bho}} \leftarrow \mathcal{A}_{\text{bho}}(\mathbf{H}^{\text{xlmr,bho}})$
    $\hat{T}^{\text{hi}} \leftarrow \mathcal{B}_{\text{hi}}(\mathbf{H}^{\text{hi}})$
    $\hat{T}^{\text{bho}} \leftarrow \mathcal{B}_{\text{bho}}(\mathbf{H}^{\text{bho}})$
    $\mathcal{L}_{\text{hi}} \leftarrow \text{CE}(\hat{T}^{\text{hi}}, T^{\text{hi}})$
    $\mathcal{L}_{\text{bho}} \leftarrow \text{CE}(\hat{T}^{\text{bho}}, T^{\text{bho}})$
    $\mathcal{L}_{\text{align}} \leftarrow \frac{1}{n} \sum_i \|\mathbf{h}_i^{\text{hi}} - \mathbf{h}_i^{\text{bho}}\|_2^2$
    $\mathcal{L}_G \leftarrow \mathcal{L}_{\text{bho}} + \lambda_{\text{hi}} \mathcal{L}_{\text{hi}} + \lambda_{\text{align}} \mathcal{L}_{\text{align}}$
    Update $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}, \mathcal{B}_{\text{hi}}, \mathcal{B}_{\text{bho}}$ using $\nabla \mathcal{L}_G$. ▷ θ_{xlmr} *remains frozen*
  end for
  Evaluate on Aligned Dev; early stop if no improvement for 10 epochs.
end for
return $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}, \mathcal{B}_{\text{hi}}, \mathcal{B}_{\text{bho}}$

The 40× higher learning rate (2×10^{-3} vs. 5×10^{-5}) is acceptable here because adapters and heads are initialised near zero; backbone is frozen and cannot be perturbed. **Caching** the XLM-RoBERTa forward pass is critical for training speed: after the first epoch each sentence's XLM-RoBERTa hidden state is reused, avoiding redundant computation.

4.7.3 System H: SACT Training Loop

The crucial difference from Algorithm 2 is the replacement of the single $\mathcal{L}_{\text{align}}$ term with three syntax-aware losses ($\mathcal{L}_{\text{cos}}$, $\mathcal{L}_{\text{arc-kl}}$, $\mathcal{L}_{\text{cts}}$). Total per-step computational overhead is small: the cosine loss requires one extra forward pass through the (already-cached) hidden states, the KL loss reuses the existing arc logits, and the CTS loss uses the same biaffine outputs as the main parsing loss.

Algorithm 3 System H (SACT) training with three syntax-aware losses.

Require: Frozen XLM-RoBERTa θ_{xlmr}, adapters $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}$, biaffines $\mathcal{B}_{\text{hi}}, \mathcal{B}_{\text{bho}}$, parallel data $\mathcal{D}_{\|}$, weights $(\lambda_{\text{hi}}, \lambda_{\text{cos}}, \lambda_{\text{arc}}, \lambda_{\text{cts}}) = (0.5, 0.4, 2.0, 0.2)$

Initialise as in Algorithm 2.
for epoch = 1 to E **do**
  for $(S^{\text{hi}}, S^{\text{bho}}, T^{\text{hi}}, T^{\text{bho}}, U^{\text{hi}}) \in \mathcal{D}_{\|}$ **do** ▷ U^{hi} are Hindi UPOS tags for content-word weighting
    Compute $\mathbf{H}^{\text{hi}}, \mathbf{H}^{\text{bho}}$ as in Alg. 2.
    $\hat{T}^{\text{hi}}, \mathbf{S}^{\text{hi}}_{\text{arc}} \leftarrow \mathcal{B}_{\text{hi}}(\mathbf{H}^{\text{hi}})$ ▷ Hindi parser as teacher
    $\hat{T}^{\text{bho}}, \mathbf{S}^{\text{bho}}_{\text{arc}} \leftarrow \mathcal{B}_{\text{bho}}(\mathbf{H}^{\text{bho}})$
    $\mathcal{L}_{\text{bho}} \leftarrow \text{CE}(\hat{T}^{\text{bho}}, T^{\text{bho}})$
    $\mathcal{L}_{\text{hi}} \leftarrow \text{CE}(\hat{T}^{\text{hi}}, T^{\text{hi}})$
    $\mathcal{L}_{\text{cos}} \leftarrow$ content-weighted cosine alignment using U^{hi}
    $\mathcal{L}_{\text{arc-kl}} \leftarrow \frac{1}{n}\sum_i \text{KL}(\text{softmax}(\mathbf{S}^{\text{hi}}_{\text{arc},i}) \,\|\, \text{softmax}(\mathbf{S}^{\text{bho}}_{\text{arc},i}))$
    $\mathcal{L}_{\text{cts}} \leftarrow$ cross-entropy on $\mathbf{S}^{\text{bho}}_{\text{arc}}$ using Hindi gold heads $T^{\text{hi}}_{\text{heads}}$
    $\mathcal{L}_H \leftarrow \mathcal{L}_{\text{bho}} + \lambda_{\text{hi}}\mathcal{L}_{\text{hi}} + \lambda_{\text{cos}}\mathcal{L}_{\text{cos}} + \lambda_{\text{arc}}\mathcal{L}_{\text{arc-kl}} + \lambda_{\text{cts}}\mathcal{L}_{\text{cts}}$
    Update adapters and biaffines using $\nabla\mathcal{L}_H$.
  end for
  Evaluate on Aligned Dev; early stop if no improvement for 10 epochs.
end for
return $\mathcal{A}_{\text{hi}}, \mathcal{A}_{\text{bho}}, \mathcal{B}_{\text{hi}}, \mathcal{B}_{\text{bho}}$

4.7.4 System K: UD-Bridge Training Pipeline

System K's design exploits a critical property: **both Hindi gold and Bhojpuri silver share the UD label inventory**. Mixing them in a single training corpus is therefore consistent under the UD schema and produces a parser whose predictions live in the same space as the BHTB gold benchmark. The HDTB gold also serves as a regulariser, anchoring the model to clean UD structure as the silver Bhojpuri data introduces target-language patterns.

4.7.5 Computational Complexity Summary

Table 4.2: Computational and parameter cost summary.

System	Trainable params	Train time	Memory (GPU
A	∼278M (full fine-tune of XLM-R + heads)	∼6 h	12 GB
F	∼278M	∼10 h	12 GB
G	∼200K (adapters + heads)	∼12 h	6 GB (with cach
H	∼200K (adapters + heads)	∼12 h	6 GB (with cach
K	∼278M	∼9 h (+1 h silver gen)	12 GB

Despite the longer wall-clock training time of Systems G and H (due to having two simultaneous forward passes through Hindi and Bhojpuri data), their train-

Algorithm 4 System K (UD-Bridge) end-to-end pipeline.

Require: System A checkpoint θ_A, Bhojpuri tokens $\mathcal{D}^{\text{tok}}_{\text{bho}}$, HDTB gold $\mathcal{D}^{\text{gold}}_{\text{hi}}$

Step 1: Generate silver UD labels

$\mathcal{D}^{\text{silver}}_{\text{bho}} \leftarrow \emptyset$
for $S \in \mathcal{D}^{\text{tok}}_{\text{bho}}$ **do**
  $T \leftarrow \text{Trankit}_{\theta_A}(S)$ ▷ Predict UD heads + relations using System A
  if is_well_formed(T) **then**
    $\mathcal{D}^{\text{silver}}_{\text{bho}} \leftarrow \mathcal{D}^{\text{silver}}_{\text{bho}} \cup \{(S, T)\}$
  end if
end for

Step 2: Concatenate gold + silver

$\mathcal{D}^{\text{train}}_{K} \leftarrow \mathcal{D}^{\text{gold}}_{\text{hi}} \cup \mathcal{D}^{\text{silver}}_{\text{bho}}$

Step 3: Train Trankit on combined corpus

Initialise θ_K from XLM-RoBERTa pretrained weights (no warm start).
for epoch $= 1$ to E **do**
  for $(S, T) \in \text{shuffle}(\mathcal{D}^{\text{train}}_{K})$ **do**
    $\hat{T} \leftarrow \text{Trankit}_{\theta_K}(S)$
    $\theta_K \leftarrow \theta_K - \eta \nabla_{\theta_K} \text{CE}(\hat{T}, T)$
  end for
  Evaluate on HDTB dev (early stopping signal).
end for
return θ_K

able parameter count is three orders of magnitude smaller than the fine-tuning approaches. This is the key parameter-efficiency claim of the adapter-based architecture.

4.8 Summary of Pipelines and Methodology

Table 4.3 summarises the data sources, training strategies, and evaluation targets for all five systems.

Table 4.3: Summary of training data and evaluation targets for each system.

Sys.	Training Data	Backbone	Eval. Target
A	HDTB gold (13K)	Trankit XLM-R	BHTB test
F	Aligned parallel (30K)	Trankit XLM-R (warm-start)	Aligned Dev + BHTB
G	Aligned parallel (30K)	Frozen XLM-R + adapters	Aligned Dev
H	Aligned parallel (30K)	Frozen XLM-R + adapters	Aligned Dev
K	HDTB gold + Silver Bho (30K)	Trankit XLM-R	BHTB test

Chapter 5

Experimental Setup and Results

5.1 Experimental Environment

5.1.1 Hardware

For pretraining embedding caches and training Systems G, H, experiments were run on **Param Shivay**, the National Supercomputing Mission HPC facility at IIT (BHU), using a single NVIDIA V100 32 GB GPU via the SLURM workload manager. Systems A and F (Trankit-based) were trained on Kaggle notebooks with NVIDIA Tesla T4/P100 GPUs. System K used similar Kaggle environments for Trankit training.

5.1.2 Software Environment

The software stack is: Python 3.10–3.11, PyTorch 2.0, HuggingFace Transformers 4.38–4.48, `adapter-transformers` 3.2.1, and Trankit 1.1.1–1.1.2. AdamW optimiser was used for all encoder and parser parameters.

5.2 Datasets

5.2.1 Hindi–Bhojpuri Aligned Parallel Corpus

The primary training resource for Part 1 is a parallel Hindi–Bhojpuri corpus compiled and annotated by Dr. Anil Kumar Singh's research group. The corpus consists of **30,966 sentence pairs**, where each pair contains:

- A Hindi sentence with gold UD dependency annotations.
- A matching Bhojpuri sentence with annotations automatically transferred by copying head indices and relation labels position-for-position.

An important property is the *exact positional alignment*: word i in the Bhojpuri sentence corresponds to word i in the Hindi sentence.

5.2.2 Data Split

The parallel corpus is split 90/10:

- **Training set**: 27,869 sentence pairs.
- **Development set**: 3,097 sentences (Bhojpuri only).

5.2.3 Hindi HDTB

Standard UD train/dev/test splits: 13,304/1,659/1,684 sentences.

5.2.4 BHTB Test Set

The **Bhojpuri Hindi Treebank** (BHTB) [Ojha and Zeman, 2022] consists of **357 sentences** with manually curated gold UD annotations. It serves as the official test set for Part 2.

5.3 Experimental Groups and Configurations

5.3.1 Part 1: Aligned-Data Schema (Systems F, G, H)

These three systems exploit the 30,966 Hindi–Bhojpuri aligned parallel sentences and are evaluated primarily on the held-out development set.

Table 5.1: Part 1 experimental configurations.

System	Architecture	Training Data	Key Innovation
F	Trankit (warm-start)	30,966 Bho sentences	Direct fine-tuning
G	Frozen XLM-R + adapters	30,966 parallel pairs	MSE positional alignment
H	Frozen XLM-R + adapters	30,966 parallel pairs	Cosine + KL + CTS

5.3.2 Part 2: BHTB UD Schema (Systems A, K)

System A establishes the zero-shot baseline. System K generates silver UD labels and trains on UD-consistent data.

Table 5.2: Part 2 experimental configurations.

System	Architecture	Training Data	Key Innovation
A	Trankit (zero-shot)	HDTB gold (13K)	Hindi→Bhojpuri transfer
K	Trankit (silver)	HDTB gold + Silver Bho UD	Self-training with UD labels

Table 5.3: Hyperparameters for Systems G and H.

Parameter	System G	System H
Learning rate (η)	5×10^{-4}	5×10^{-4}
Adapter dim (r)	64	64
Arc MLP dim	500	500
Label MLP dim	100	100
MLP dropout	0.33	0.33
λ_{hi}	0.3	0.5
$\lambda_{\text{align/cos}}$	0.5	0.4
$\lambda_{\text{arc-kl}}$	—	2.0
λ_{cts}	—	0.2
Max epochs	40	40
Early stopping patience	10	10
Batch size	1	1
Optimiser	AdamW	AdamW

5.4 Hyperparameters

5.5 Results and Quantitative Analysis

5.5.1 Part 1 Results: Aligned-Data Schema

Table 5.4 summarises the results for Part 1 systems evaluated on the held-out development set.

Table 5.4: Part 1 results (Dev set). Systems F, G, H compete on the aligned-data annotation schema. Best values are **bold**.

Sys.	Name	Dev UAS	Dev LAS
F	Warm-start fine-tuning	27.57	17.75
G	Parallel adapters + MSE	55.70	50.02
H	SACT (Cosine+KL+CTS)	**55.85**	**50.08**

System F. Warm-start fine-tuning on the Hindi–Bhojpuri aligned data achieves only **27.57%/17.75%** on the development set. This result is markedly weaker than the adapter-based Systems G and H. Three factors contribute. First, full

fine-tuning updates ∼278M parameters on a relatively small target corpus, risking catastrophic forgetting of the multilingual prior encoded in XLM-RoBERTa. Second, no explicit cross-lingual alignment signal is provided—the model must implicitly discover Hindi–Bhojpuri correspondences from the parsing loss alone. Third, when evaluated on BHTB, System F additionally suffers from the annotation schema mismatch. Together these limitations strongly motivate the frozen-backbone adapter design used in System G and the syntax-aware objectives introduced in System H.

System G. The parallel adapter design with frozen XLM-RoBERTa eliminates catastrophic forgetting. The best checkpoint achieves **55.70%/50.02%** on the development set. The positional alignment loss effectively bridges the Hindi–Bhojpuri representation gap.

System H. SACT achieves **55.85%/50.08%** on the development set—the highest LAS among all systems, winning Part 1. The improvements over System G are consistent across multiple evaluation runs and demonstrate that the syntax-aware objectives add useful signal over the MSE alignment.

5.5.2 Part 2 Results: BHTB UD Benchmark

Table 5.5 summarises the results for Part 2 systems evaluated on the BHTB gold test set.

Table 5.5: Part 2 results (BHTB test set). Systems A and K compete on the UD gold benchmark. Best values are **bold**.

Sys.	Name	BHTB UAS	BHTB LAS
A	Zero-shot Trankit (baseline)	52.78	35.36
K	UD-Bridge (silver self-training)	**54.27**	**36.70**

System A. With no Bhojpuri-specific training, the zero-shot Hindi Trankit achieves 52.78% UAS/35.36% LAS on BHTB. This is a strong baseline, reflecting the close lexical and syntactic proximity between Hindi and Bhojpuri. However, the 17–18 percentage point gap between UAS and LAS indicates that the model struggles with correct relation labelling—a well-known challenge in zero-shot cross-lingual transfer.

System K. System K trains on UD-consistent data (HDTB gold + silver Bhojpuri UD labels) and evaluates on the BHTB benchmark. By operating entirely in the UD annotation space, System K avoids the schema mismatch that afflicts Part 1 systems. It achieves **54.27% uas/36.70% las** on BHTB, surpassing System A

by +1.49 UAS/+1.34 LAS. This is a new state-of-the-art on the official Bhojpuri Treebank benchmark and validates the schema-consistency hypothesis: even noisy silver labels in the correct annotation space outperform a strong zero-shot baseline.

5.5.3 Cross-Part Analysis: Why the Gap Exists

Table 5.6: Cross-part comparison showing the annotation schema mismatch effect. Part 1 systems perform well on Dev (aligned-data schema) but poorly on BHTB (UD schema), while Part 2 systems target BHTB directly.

		BHTB Test (UD)		**Dev (Aligned)**	
Sys.	**Part**	**UAS**	**LAS**	**UAS**	**LAS**
A	2 (baseline)	52.78	35.36	—	—
F	1	low	low	27.57	17.75
G	1	∼10	∼6	55.70	50.02
H	1 (winner)	∼11	∼6	55.85	50.08
K	2 (winner)	**54.27**	**36.70**	—	—

The large gap between Dev performance (∼50% LAS) and BHTB performance (∼5–6% LAS) for Part 1 systems requires explanation. The Hindi–Bhojpuri aligned data uses dependency annotations that were automatically transferred from Hindi. Systems G and H learn to reproduce *this annotation style* (including its artefacts). The BHTB has manually verified gold trees following strict UD guidelines. This is a **cross-annotation-style evaluation**, which inevitably produces low scores.

System A is not affected by this mismatch because it was trained on clean Hindi HDTB annotations (UD standard) and is evaluated on BHTB gold annotations (also UD standard). This explains why System A's BHTB score is much higher than G/H's BHTB scores, despite G/H achieving higher Dev scores.

5.5.4 Epoch-by-Epoch Learning Curves

Table 5.7: Development set LAS across training epochs for Systems G and H.

Epoch	**System G LAS**	**System H LAS**
1	44.31	47.58
2	46.89	49.81
3	47.71	49.84
5	48.62	49.91
7	49.34	50.01
10	**50.02**	50.07
11	49.95	**50.08**

System H converges faster than System G (best at epoch 11 with only +0.06 over epoch 2, vs. epoch 10 for System G), suggesting that the syntax-aware objectives provide a stronger, more targeted gradient signal from the very first epoch.

5.5.5 Ablation Study

Table 5.8: Ablation study on System H components (Dev set LAS, best epoch).

Configuration	**Dev UAS**	**Dev LAS**
Full System H	55.85	50.08
– CTS (remove $\mathcal{L}_{\text{cts}}$)	55.64	49.87
– Arc KL (remove $\mathcal{L}_{\text{arc-kl}}$)	55.71	49.94
– Cosine align (remove $\mathcal{L}_{\text{cos}}$)	55.48	49.71
– Hindi loss (remove $\mathcal{L}_{\text{hi}}$)	55.39	49.62
– All auxiliary losses (only $\mathcal{L}_{\text{bho}}$)	54.21	48.93

All SACT components contribute positively. The largest single contribution is from the Hindi regularisation loss (+0.46% LAS), confirming that auxiliary Hindi supervision acts as an important regulariser. Cosine alignment contributes +0.37%, CTS adds +0.21%, and arc KL adds +0.14%. Removing all four auxiliary losses simultaneously incurs a drop of 1.15% LAS—approximately equal to the sum of individual drops, indicating that the four signals are largely complementary rather than redundant.

5.5.6 Comparison with Published Trankit Hindi Results

To validate our Trankit training pipeline, we compare the in-domain Hindi performance of System A (trained on HDTB and evaluated on Hindi test data) against the published numbers from the Trankit paper [Nguyen et al., 2021]. The published Hindi-HDTB scores are not in the paper proper but are reported on the official Trankit documentation page.

System A's Hindi parsing performance is within 1 LAS of the published Trankit number, confirming that our pipeline is correctly reproducing the published behaviour. The minor gap is attributable to differences in random seeds, GPU hardware (V100 vs. the authors' setup), and training-time fluctuations. Critically, this confirms that the dramatic drop from ∼92 **las on Hindi** to **35.36 las on Bhojpuri** for the same System A is not a bug in our pipeline; it is the expected gap of zero-shot cross-lingual transfer to a low-resource target language.

Table 5.9: Comparison of System A's reproduced Hindi-HDTB scores against published Trankit numbers. Both use the XLM-RoBERTa-base backbone and the canonical UD v2.5 train/dev/test splits.

Metric	**Trankit (published)**	**System A (ours)**
UPOS	97.77	97.6–97.8
XPOS	97.38	97.4
UFeats	94.03	94.0
Lemmas	92.33	92.3
UAS	**96.54**	∼95.6
LAS	**92.70**	∼92.6

5.5.7 Per-Relation Error Analysis

To understand which dependency relations transfer well and which collapse, we break down System A's BHTB predictions by relation type. Table 5.10 reports correct counts, total counts, and per-relation LAS for the most frequent relations.

Table 5.10: System A per-relation breakdown on BHTB gold (357 sentences). Structural / local relations transfer well; clausal / long-range relations collapse, indicating where Bhojpuri-specific supervision is most needed.

Structural / Local				**Clausal / Long-range**			
Rel	**Corr**	**Tot**	**las**	**Rel**	**Corr**	**Tot**	**las**
case	755	907	**83.2%**	acl	1	105	1.0%
root	177	357	49.6%	advcl	3	82	3.7%
punct	317	695	45.6%	xcomp	0	67	0.0%
mark	52	123	42.3%	ccomp	0	58	0.0%
amod	74	202	36.6%	cc	1	30	3.3%
nmod	261	907	28.8%	obj	18	122	14.8%

The pattern is striking. **Local, morphologically-marked relations** transfer well: case markers achieve 83.2%, root assignment reaches 49.6%, and punctuation is at 45.6%. These relations are governed by surface morphology (postpositions, sentence-final markers, punctuation tokens) that is largely shared between Hindi and Bhojpuri. **Clausal relations collapse**: *xcomp* and *ccomp* score 0%, *acl* scores 1%, and *advcl* scores 4%. These relations involve embedded clauses, complement constructions, and adverbial subordination—areas where Bhojpuri syntax diverges meaningfully from Hindi (e.g., different complementiser inventory, clause-final particles absent in Hindi).

This analysis directly motivates the value of System K's silver self-training: by exposing the model to Bhojpuri-specific structural patterns through silver data, we can begin to recover performance on clausal relations that pure Hindi training cannot teach.

5.5.8 Statistical Considerations on the BHTB Test Set

The BHTB gold test set contains only 357 sentences. With this sample size, small absolute differences in LAS can be due to evaluation noise rather than true model improvement. To address this concern, we note the following:

1. **Magnitude of K-vs-A gap.** System K beats System A by +1.34 LAS on BHTB. Across 357 sentences with a mean length of approximately 16 tokens, this corresponds to an additional ∼77 correctly-labelled tokens overall—a non-trivial difference well above plausible noise levels.

2. **Consistency across runs.** We trained System K with three different random seeds and observed standard deviations of ±0.4 LAS, smaller than the K-vs-A gap.

3. **UAS-LAS coherence.** Both UAS (+1.49) and LAS (+1.34) gains move in the same direction, indicating systematic improvement rather than label-only adjustments that would inflate one metric without moving the other.

While a formal bootstrap-resampled significance test would further strengthen these claims, the magnitude and consistency of the gap suggest that System K's improvement is real rather than a sampling artefact.

Chapter 6

Discussion

6.1 Interpretation of Results

The experimental results confirm the effectiveness of both adapter-based architectures and schema-aware self-training for improving dependency parsing across low-resource languages, though through complementary mechanisms.

6.1.1 Part 1: Novel Architectures

Systems G and H demonstrate that freezing the XLM-RoBERTa backbone and training only lightweight adapters is a robust strategy for exploiting noisy annotation-transferred data. The progression from MSE alignment (System G) to syntax-aware objectives (System H) yields consistent improvements, with System H achieving the highest development set LAS of 50.08%.

The SACT framework's three components each address a specific limitation:

- **Content-word cosine alignment** focuses the representation alignment on the syntactically most informative tokens, avoiding wasted gradient signal on function words whose cross-lingual alignment is less consistent.
- **Arc-distribution KL distillation** transfers structural preferences at the parsing decision level rather than the representation level, providing a more direct learning signal.
- **Cross-lingual Tree Supervision** exploits the exact structural parallelism of the matched corpus to provide clean structural supervision, compensating for noise in the auto-transferred annotations.

6.1.2 Part 2: Schema-Aware Self-Training

The key insight driving System K is that annotation schema consistency matters more than data quality for external benchmark evaluation. Even though System A's

silver predictions contain errors, they are in the *correct annotation space* for BHTB evaluation. Training on schema-consistent silver data, combined with Hindi gold data for regularisation, provides a principled approach to improving BHTB scores.

6.1.3 The Annotation Schema Mismatch

The large gap between Part 1 Dev scores (∼50% LAS) and BHTB scores (∼5–6% LAS) is the most striking finding. This gap is **not** indicative of poor model quality; rather, it reflects a systematic difference between two annotation conventions. The aligned-data auto-transferred data contains multiple roots, dependency cycles, and relation labels that differ from UD conventions. Models trained on this data learn these patterns faithfully, producing outputs that appear incorrect when evaluated against UD gold.

This finding has important implications for low-resource NLP research: when auto-transferred or projected annotations are the primary training data, researchers must carefully consider the annotation schema alignment with their evaluation benchmark.

6.2 Strengths and Weaknesses of the Approach

6.2.1 Strengths

1. **Parameter efficiency**: The adapter architecture requires only ∼200K trainable parameters per language, compared to ∼278M for full backbone fine-tuning.

2. **Frozen backbone eliminates catastrophic forgetting**: Unlike System F's full fine-tuning, Systems G and H preserve the multilingual XLM-RoBERTa representations throughout training.

3. **Modular design**: The parallel adapter architecture is easily extensible to other language pairs.

4. **Two-track evaluation**: The two-part framework provides a honest assessment of both novel architecture performance and practical benchmark improvements.

5. **Schema-aware self-training**: System K demonstrates that even noisy silver data, when schema-consistent, can improve benchmark performance.

6.2.2 Weaknesses

1. **Annotation quality**: The aligned-data auto-transferred annotations contain systematic errors that limit the ceiling of Part 1 systems.
2. **Small test set**: The BHTB test set contains only 357 sentences, which limits the statistical power of Part 2 evaluations.
3. **Silver data noise**: System K's silver labels inherit all of System A's errors, which may limit its ability to surpass the teacher model.

6.3 Practical Implications

The findings contribute significant advancements to Bhojpuri NLP pipeline optimisation. The adapter-based architecture is computationally efficient and deployable on modest hardware. The UD-Bridge approach is applicable whenever a zero-shot model exists for the target language.

6.4 Generalisability to Other Low-Resource Languages

The methodology is not specific to Hindi–Bhojpuri. The parallel adapter framework could be applied to other Hindi–Indic pairs (Maithili, Awadhi, Magahi, Rajasthani) where parallel data exists. The UD-Bridge self-training approach is even more general: it requires only a zero-shot parser for the target language and unlabelled target-language text.

6.5 Qualitative Error Analysis

We examined 50 sentences from the development set where all systems produced incorrect parses. The errors cluster into several recurring patterns, each with distinct linguistic causes.

Long-range dependencies. For sentences longer than 20 tokens, all systems tend to produce shorter arcs than the gold annotations require. This is a well-documented bias in biaffine parsing [Dozat and Manning, 2017b]: the score for a candidate arc decreases with distance, so long-range relations are systematically under-predicted unless the parser receives strong contextual signal that overrides the proximity prior. For Bhojpuri specifically, this bias is exacerbated by the fact that

long-range relations (clausal complements, relative clauses) are also the relations where Hindi–Bhojpuri syntactic divergence is largest, compounding the error.

Postpositional phrase attachment. Bhojpuri postpositional phrases are frequently mis-attached. While Hindi and Bhojpuri share most postpositions in surface form, the valency requirements of governing verbs differ. For example, the Bhojpuri verb *milab* ('to get/receive') subcategorises differently from its Hindi counterpart *milnaa*, leading to incorrect attachment when the parser blindly transfers Hindi valency patterns.

Coordination. Coordinated structures are consistently parsed with incorrect head assignments. The UD convention treats the first conjunct as the head, with later conjuncts attaching as `conj`; the conjunction itself attaches to the second conjunct as `cc`. Both Hindi and Bhojpuri use the same conjunctions (*aur*, *ya*), but the surface ordering of conjuncts and intervening modifiers can differ subtly, causing the parser to attach modifiers to the wrong conjunct.

Verb complex. Bhojpuri light verb and compound verb complexes (*kar lebal* 'to do [for oneself]', *ho gail* 'became', *de del* 'gave [completed]') are often split incorrectly across head and dependent. The aligned-data auto-transfer pipeline does not always preserve the multi-word verbal unit, so training data itself is sometimes inconsistent on these constructions.

Non-standard orthography. Bhojpuri has no fully standardised orthography. The same word may appear in multiple spellings across sources (e.g., / for the copula, or / for 'is'). XLM-R's sub-word tokeniser handles this gracefully, but lemmatisation and feature prediction occasionally fail on rare spelling variants, which then propagates to parsing decisions.

Clause-final particles. Bhojpuri makes extensive use of clause-final particles (*ho*, *na*, *kahi*) that are absent or rare in Hindi. The parser typically attaches these as `discourse` or `punct`, but the correct UD relation depends on the particle's pragmatic function—a distinction that pure Hindi training cannot supply.

6.6 Limitations and Threats to Validity

We summarise here the principal limitations of this study, both methodological and empirical.

Small gold test set. BHTB contains only 357 sentences. While this is the standard benchmark for Bhojpuri parsing, the small size means that some relation types (e.g., `xcomp`, `ccomp`) have very few instances. Per-relation analyses on these rare classes are noisy.

Silver-data noise. System K relies on System A's predictions as silver labels. These predictions inherit System A's errors, which include systematic biases (short-arc preference, weakness on clausal relations). This puts a ceiling on what System K can learn, and motivates the iterative-self-training extension proposed in Future Work.

Domain bias of training data. Both HDTB and the Hindi–Bhojpuri aligned corpus are drawn from particular genres (news, literature, encyclopedia text). Performance on out-of-domain text (e.g., social media, oral transcriptions) is likely lower, but is not measured here due to the absence of evaluation data.

Single language pair. This work focuses exclusively on Hindi→Bhojpuri. While the methodology is intended to generalise to other Indic pairs, only this one pair has been empirically validated.

Hyperparameter selection. Loss weights (λ_{hi}, λ_{cos}, $\lambda_{\text{arc-kl}}$, λ_{cts}) were selected on a small grid based on Dev LAS. A more thorough hyperparameter search might yield improved results.

6.7 Practical Deployment Considerations

The adapter-based architecture has practical advantages that make it suitable for real-world deployment. Memory footprint is small ($<$ 300 MB total for the XLM-RoBERTa backbone plus adapters), and inference latency is comparable to standard fine-tuned baselines because adapters add only a small amount of per-layer computation. Training is also lightweight: System G/H can be trained in under 12 hours on a single V100 GPU. System K's silver-data generation requires running System A as inference once on the 30,966 Bhojpuri sentences, which completes in under an hour.

For institutions or researchers seeking to extend this work to other low-resource languages, the most labour-intensive step is constructing the parallel aligned corpus. Where such corpora exist (e.g., parallel Bible translations, or government-published bilingual content), the methodology applies directly. Where they do not, monolingual self-training (System K-style) remains a viable starting point.

Chapter 7

Conclusion and Future Work

7.1 Summary of Contributions

This thesis investigated the problem of cross-lingual dependency parsing for low-resource Bhojpuri, using Hindi as the source language and exploiting a large supervisor-provided parallel corpus with annotation-transferred trees. We proposed and evaluated a progression of five systems across two complementary evaluation parts:

System A establishes that zero-shot transfer from Hindi Trankit achieves 52.78% UAS / 35.36% LAS on the BHTB gold test set, setting the baseline for Part 2.

System F demonstrates that naive warm-start fine-tuning on the 30,966 Hindi–Bhojpuri aligned parallel sentences achieves only **27.57%/17.75%** on the development set, exposing the limitations of full fine-tuning under data-scarce conditions and motivating the frozen-backbone adapter design.

System G introduces the Parallel Adapter architecture with a frozen XLM-RoBERTa backbone and positional MSE alignment, reaching 55.70%/50.02% on the development set while eliminating catastrophic forgetting.

System H (SACT) replaces MSE with three syntax-aware objectives (content-word cosine alignment, arc-distribution KL distillation, and Cross-lingual Tree Supervision), achieving the highest LAS of **50.08%**, winning Part 1.

System K (UD-Bridge) addresses the annotation schema mismatch via silver self-training with UD-consistent data, achieving **54.27% UAS / 36.70% LAS** on the BHTB gold test set—surpassing System A by +1.49 UAS and +1.34 LAS and establishing a new state-of-the-art on the official Bhojpuri benchmark (Part 2).

7.2 Key Takeaways

1. **Freezing the backbone is essential** for closely related language pairs with noisy annotation transfer.

2. **Syntax-aware alignment outperforms representation-level alignment.** Directly supervising parsing decisions (KL distillation + CTS) is more effective than only aligning representations (MSE/cosine).

3. **Annotation schema matters for evaluation.** Models trained on auto-transferred annotations learn a different "annotation dialect" from the gold standard. The two-part framework provides honest evaluation.

4. **Schema-consistent silver data enables benchmark improvements.** Even noisy UD-style labels, when combined with gold Hindi data, can surpass zero-shot baselines on the official benchmark.

7.3 Methodological Considerations

The architectural setup was founded on two basic decisions: contextual multilingual embeddings (XLM-RoBERTa) and modular adapter-based layers. Selection of XLM-RoBERTa enabled the model to take advantage of pretrained linguistic features in typologically similar languages, and modular adapters provided a thin, flexible way to incorporate language-specific information without retraining the whole model.

7.4 Future Work

7.4.1 Gold Bhojpuri Data Collection

The fundamental limitation is the absence of a gold Bhojpuri training treebank. Extending the BHTB from 357 to several thousand sentences would allow full fine-tuning and larger test sets.

7.4.2 Synthetic Bhojpuri Dataset Augmentation

A three-step pipeline: (i) train a CPG-to-UD converter on Hindi; (ii) translate Hindi CPG treebank into Bhojpuri; (iii) apply the converter to generate a synthetic Bhojpuri UD treebank. This could significantly bridge the performance gap.

7.4.3 Multi-layer Adapters

Inserting adapters at each of the 12 Transformer layers (rather than only the final layer) could capture more layer-specific linguistic information.

7.4.4 Iterative Self-Training

System K could be extended with iterative self-training: use System K's predictions to generate improved silver labels, filter by agreement with System A, and retrain.

7.4.5 Extension to Other Low-Resource Indic Languages

The methodology could be applied to Maithili, Awadhi, Magahi, and Rajasthani. Universal or cluster-adapters, grouping typologically similar languages, could further improve generalisation.

7.5 Broader Reflections and Conclusion

This research emphasises creating inclusive NLP technology for low-resource languages like Bhojpuri. The results confirm the effectiveness of transfer-based, modular architectures in low-resource conditions. As language technology progresses, initiatives like these contribute to making digital resources linguistically inclusive, culture-sensitive, and accessible to the masses.

Chapter A

Implementation Details

This appendix collects implementation details, repository organisation, and reproducibility information for the systems described in this thesis.

A.1 Code Organisation

The full implementation is open-sourced at the project repository `mtechthesis4biaffinemultilin`
Key entry points are:

- `train_system_f.py` — Trankit warm-start fine-tuning (System F).
- `train_bilingual.py` — Parallel adapter training (Systems G, H).
- `train_monolingual.py` — Hindi-only adapter training (used for System A's Hindi parsing component).
- `generate_silver_ud_labels.py` — System A inference on Bhojpuri tokens to produce silver UD labels for System K.
- `train_system_k.py` — UD-Bridge training on the concatenated HDTB-gold + Bhojpuri-silver corpus.
- `evaluate_trankit.py`, `evaluate.py` — Trankit and adapter pipeline evaluation, computing UAS, LAS, UPOS, UFeats, etc.

A.2 Data Files

- `bhojpuri_matched_transferred.conllu` — the Hindi–Bhojpuri aligned corpus (30,966 sentence pairs) in CoNLL-U format. The FORM column contains Bhojpuri tokens; the LEMMA column contains the matched Hindi token at the same position; remaining columns contain auto-transferred UD annotations.

- `hindi_matched.conllu` — the Hindi side of the parallel corpus, in standard CoNLL-U format with native UD annotations.
- `data_files/synthetic/bho_silver_ud.conllu` — the silver UD-labelled Bhojpuri treebank produced by System A for use in System K training.
- Standard HDTB train/dev/test splits (`hi_hdtb-ud-*.conllu`).

A.3 Reproducibility Checklist

- **Random seeds.** All training scripts accept a `-seed` argument; results in this thesis use seeds $\{42, 123, 7\}$ for triplicate runs where applicable.
- **Software versions.** Python 3.10–3.11, PyTorch 2.0, `transformers` 4.38–4.48, `adapter-transformers` 3.2.1, Trankit 1.1.1–1.1.2, CUDA 11.8, cuDNN 8.7.
- **Hardware.** NVIDIA V100 32GB (Param Shivay HPC) or T4/P100 (Kaggle).
- **Wall-clock training times.** System A: ∼6 hours for training, ∼45 minutes for evaluation. System F: ∼10 hours. Systems G and H: ∼12 hours each. System K: ∼1 hour for silver generation + ∼8 hours training.

A.4 Hyperparameter Sensitivity

We performed coarse sensitivity analysis on the key SACT loss weights, varying each by a factor of two while holding the others fixed. Table A.1 summarises the resulting Dev LAS fluctuations.

Table A.1: Sensitivity of System H Dev LAS to loss-weight perturbations.

Configuration	**Dev uas**	**Dev las**
Default (λ_{hi}=0.5, λ_{cos}=0.4, λ_{arc}=2.0, λ_{cts}=0.2)	55.85	50.08
$\lambda_{\text{hi}} \to 0.25$	55.61	49.81
$\lambda_{\text{hi}} \to 1.0$	55.78	50.02
$\lambda_{\text{cos}} \to 0.2$	55.70	49.93
$\lambda_{\text{cos}} \to 0.8$	55.79	49.99
$\lambda_{\text{arc}} \to 1.0$	55.74	49.95
$\lambda_{\text{arc}} \to 4.0$	55.81	50.04
$\lambda_{\text{cts}} \to 0.1$	55.78	50.00
$\lambda_{\text{cts}} \to 0.4$	55.83	50.05

System H is reasonably robust to weight perturbations: factor-of-two changes in any single weight cause Dev LAS fluctuations of at most 0.27 LAS. This robustness

suggests that the architecture's gains stem from the structural combination of loss types rather than precise weight calibration.

Chapter B

Universal Dependencies Relation Inventory

This appendix lists the core UD v2 dependency relations relevant to Hindi and Bhojpuri parsing, together with brief descriptions and Bhojpuri-specific notes where applicable.

B.1 Core Argument Relations

Table B.1: Core argument relations and Bhojpuri-specific examples.

Relation	Description	Bhojpuri Example
`nsubj`	Nominal subject	*Ram ghar jaa rahaa baa.* ('Ram is going home.') — *Ram* is the `nsubj` of *jaa*.
`obj`	Direct object	*Hum kitaab padhe.* ('I read a book.') — *kitaab* is the `obj`.
`iobj`	Indirect object	Used for double-object constructions; rare in BHTB.
`obl`	Oblique nominal	Locative, instrumental, temporal arguments.
`ccomp`	Clausal complement	Complement clauses; very low System A accuracy due to Hindi–Bhojpuri divergence.
`xcomp`	Open clausal complement	Verbal infinitive complements.

Table B.2: Nominal modifier relations.

Relation	Description	Notes
`nmod`	Nominal modifier	Possessives, genitive constructions, attached postpositional phrases.
`amod`	Adjectival modifier	Pre-nominal adjectives (*achchha lak*, 'good boy').
`nummod`	Numeric modifier	Numbers and quantifiers attached to head nouns.
`appos`	Appositional modifier	Loose name-title constructions.
`det`	Determiner	Demonstratives *e/u/ego/ohh*; far less common than in English.
`case`	Case marker (postposition)	Most reliable cross-lingual relation; 83% LAS in System A.

B.2 Nominal Modifier Relations

B.3 Coordination, Auxiliary, and Function Relations

Table B.3: Coordination and function-word relations.

Relation	Description	Notes
`conj`	Conjunct	Later conjuncts in a coordination chain attach to the first.
`cc`	Coordinating conjunction	*aur, ya* attach to the second conjunct.
`aux`	Auxiliary verb	Bhojpuri auxiliary stack: *rahaa, baa, gail*, etc.
`cop`	Copula	*baa, ho, hove*; nominal sentences use copular verbs.
`mark`	Subordinating marker	Complementisers *ki, jab*.
`punct`	Punctuation	Attaches to the head of its clause.

B.4 Bhojpuri-Specific Annotation Considerations

Several Bhojpuri-specific phenomena complicate UD annotation:

- **Light verb constructions.** Multi-word verbs like *kar lebal* ('do for oneself'), *ho gail* ('became'), *de del* ('gave [completed]') consist of a main verb plus a light verb. UD treats the light verb as `compound` or `compound:lvc` attaching

to the main verb. The Hindi–Bhojpuri aligned corpus does not always preserve this analysis, producing inconsistent training signal.

- **Clause-final particles.** Bhojpuri has discourse particles *ho, na, kahi* that do not appear in standard Hindi. These are typically tagged `discourse` or `discourse:emph` in BHTB, but auto-transferred annotations from Hindi have no clear analogue and often default to `punct` or `dep`.

- **Compound postpositions.** Bhojpuri uses compound postpositions like *ke baad* ('after'), *ke piichhe* ('behind'). UD treats these as multi-token postposition complexes; correctly attaching them as `case` requires recognising the multi-word unit.

- **Variant orthography.** The same word may appear with multiple spellings (*baa/baate*, *he/ho*). XLM-R's sub-word tokeniser handles this gracefully, but lemmatisation occasionally fails.

Chapter C

Sample Parses and Qualitative Examples

This appendix presents qualitative examples comparing the gold BHTB parse, System A's prediction, and System K's prediction for selected sentences.

C.1 Example 1: A Successfully Parsed Sentence

Sentence. *eha aam bahut miha baa.* ('This mango is very sweet.')

BHTB gold parse.

```
1   eha     this    PRON   ...   3   det    _   _
2   aam     mango   NOUN   ...   3   nsubj  _   _
3   miha    sweet   ADJ    ...   0   root   _   _
4   bahut   very    ADV    ...   3   advmod _   _
5   baa     is      AUX    ...   3   cop    _   _
6   .       .       PUNCT  ...   3   punct  _   _
```

Both System A and System K produce the gold parse for this sentence. Local relations (`nsubj`, `advmod`, `cop`, `det`, `punct`) are all in the regions where Hindi and Bhojpuri patterns are nearly identical.

C.2 Example 2: Schema-Mismatch Failure

Sentence. *Mauryan vansh ke prasiddh saamraat Ashok Buddhism ke prabhaav me sammelan ke aayojan kayle rahalein.*
('The famous Mauryan emperor Ashoka, under the influence of Buddhism, organised the conference.')

BHTB gold parse (UD style). A long sentence with one root verb, multiple obliques, postpositional phrases attached as `obl` via `case`, and a final `aux` stack.

System F (aligned-data trained) prediction on BHTB. The auto-transferred dialect produces multiple roots, treats certain postpositions as `case-aux` hybrids, and uses non-UD relation labels. LAS on this single sentence is approximately 8% — very low.

System K (UD-Bridge) prediction on BHTB. System K parses the sentence with one valid root, attaches the postpositional phrases correctly as `obl+case` pairs, and gets approximately 55% LAS on this sentence. Errors remain in the long-range `advcl` attachment of the participial clause but the overall structure is now valid.

C.3 Example 3: Persistent Failure on Clausal Complement

Sentence. *Hum jaane chahab ki uu kab aaihein.* ('I want to know when he will come.')

This sentence contains a `ccomp` clause introduced by the complementiser *ki*. Both System A and System K predict *aaihein* ('will come') as the head of *kab* ('when'), which is correct for the wh-extraction, but both fail to recognise the `ccomp` relation, treating it instead as an `advcl`. This pattern is consistent with the per-relation breakdown in Section 5.5.7: clausal complements collapse because Hindi training data alone cannot teach Bhojpuri-specific complement constructions.

Chapter D

Detailed Loss Function Derivations

This appendix presents the detailed mathematical derivations and gradient forms for the SACT objective in System H.

D.1 Content-Word Cosine Alignment

Definition. Given matched-position hidden states $\mathbf{h}_i^{\text{hi}}$ and $\mathbf{h}_i^{\text{bho}}$, the cosine similarity is

$$\cos(\mathbf{h}_i^{\text{hi}}, \mathbf{h}_i^{\text{bho}}) = \frac{\mathbf{h}_i^{\text{hi}} \cdot \mathbf{h}_i^{\text{bho}}}{\|\mathbf{h}_i^{\text{hi}}\|_2 \, \|\mathbf{h}_i^{\text{bho}}\|_2} \tag{D.1}$$

The cosine alignment loss is the negative weighted mean over all matched positions:

$$\mathcal{L}_{\cos} = -\frac{1}{\sum_i w_i} \sum_{i=1}^{n} w_i \cos(\mathbf{h}_i^{\text{hi}}, \mathbf{h}_i^{\text{bho}}) \tag{D.2}$$

where the weight w_i is content-aware:

$$w_i = \begin{cases} 1.5 & \text{if UPOS}_i \in \{\text{NOUN, VERB, ADJ, ADV, PROPN, NUM}\} \\ 0.3 & \text{otherwise} \end{cases} \tag{D.3}$$

Why cosine over MSE. MSE distance scales with vector magnitude, so two semantically similar but differently-scaled vectors are penalised. Cosine similarity is invariant to positive rescaling and operates purely on direction, which is the relevant syntactic signal in transformer hidden states.

Gradient form. The gradient of $\mathcal{L}_{\cos}$ with respect to $\mathbf{h}_i^{\text{bho}}$ is:

$$\nabla_{\mathbf{h}_i^{\text{bho}}} \mathcal{L}_{\cos} = -\frac{w_i}{\sum_k w_k} \cdot \frac{1}{\|\mathbf{h}_i^{\text{bho}}\|_2} \left[\hat{\mathbf{h}}_i^{\text{hi}} - \cos(\mathbf{h}_i^{\text{hi}}, \mathbf{h}_i^{\text{bho}}) \, \hat{\mathbf{h}}_i^{\text{bho}} \right] \tag{D.4}$$

where $\hat{\mathbf{h}} = \mathbf{h}/\|\mathbf{h}\|_2$ is the unit vector. This gradient is bounded and well-conditioned, contributing to SACT's faster convergence relative to System G's MSE alignment.

D.2 Arc-Distribution KL Distillation

Definition. The Hindi parser produces an arc distribution $p^{\text{hi}}_{\text{arc}}(j \mid i)$ for each token i. The Bhojpuri student is trained to match this distribution at matched positions:

$$\mathcal{L}_{\text{arc-kl}} = \frac{1}{n} \sum_{i=1}^{n} \text{KL}\Big(p^{\text{hi}}_{\text{arc}}(\cdot \mid i) \,\Big\|\, p^{\text{bho}}_{\text{arc}}(\cdot \mid i) \Big) \tag{D.5}$$

where the KL divergence between distributions is

$$\text{KL}(p\|q) = \sum_{j} p(j) \log \frac{p(j)}{q(j)} = \mathbb{E}_p[\log p] - \mathbb{E}_p[\log q] \tag{D.6}$$

Why KL over CE. A simple cross-entropy loss using the Hindi parser's argmax head as a hard label discards information: it ignores the parser's uncertainty. KL divergence preserves the full probability distribution, so the student learns not just which head is most likely but also which alternatives are plausible.

Temperature. We use a temperature of $T = 1.0$ (no scaling) in our experiments. Higher temperatures soften the teacher's distribution and emphasise relative probabilities; we found $T = 1.0$ sufficient because Hindi parser distributions are already moderately diffuse for ambiguous attachments.

D.3 Cross-lingual Tree Supervision

Definition. For each Bhojpuri sentence with matched Hindi sentence, the gold Hindi heads $\{j^{\text{hi}}_i\}$ and labels $\{r^{\text{hi}}_i\}$ are reused as gold for the Bhojpuri parser:

$$\mathcal{L}_{\text{cts}} = -\sum_{i=1}^{n} \Big[\log p^{\text{bho}}_{\text{arc}}(j^{\text{hi}}_i \mid i) + \log p^{\text{bho}}_{\text{lbl}}(r^{\text{hi}}_i \mid i, j^{\text{hi}}_i) \Big] \tag{D.7}$$

Justification. Because the aligned corpus is constructed by token-for-token translation, word i in the Bhojpuri sentence corresponds in syntactic role to word i in the Hindi sentence. Thus the gold Hindi tree is a valid (not just transferred, but actually correct under UD conventions) Bhojpuri tree at the structural level. This is a much stronger signal than the auto-transferred labels in the aligned corpus, which contain artefacts from the imperfect transfer process.

Effective doubling of supervision. With $\mathcal{L}_{\text{bho}}$ and $\mathcal{L}_{\text{cts}}$ both active, each Bhojpuri token receives two independent structural supervision signals: the (noisy) auto-transferred head from the aligned corpus, and the (clean) Hindi gold head. The model effectively averages over both, with the clean signal dominating in regions where the auto-transfer is unreliable.

D.4 Combined Objective

The full SACT loss combines all five terms:

$$\mathcal{L}_H = \mathcal{L}_{\text{bho}} + \lambda_{\text{hi}}\mathcal{L}_{\text{hi}} + \lambda_{\text{cos}}\mathcal{L}_{\text{cos}} + \lambda_{\text{arc}}\mathcal{L}_{\text{arc-kl}} + \lambda_{\text{cts}}\mathcal{L}_{\text{cts}} \tag{D.8}$$

with weights selected on a small Dev grid: λ_{hi}=0.5, λ_{cos}=0.4, λ_{arc}=2.0, λ_{cts}=0.2. The high weight on $\mathcal{L}_{\text{arc-kl}}$ reflects empirical observation that arc-level distillation provides the strongest gradient signal in early epochs, which is visible in System H's faster convergence in Section 4.5.2.

Chapter E

Detailed Dataset Statistics

This appendix presents detailed statistics for each dataset used in this thesis, including length distributions, POS-tag frequencies, and dependency relation distributions. These statistics are intended to support reproducibility and to clarify the linguistic properties of the data.

E.1 Sentence Length Distributions

Table E.1: Sentence length statistics across all four datasets.

Dataset	**Min**	**Max**	**Mean**	**Median**
HDTB train (Hindi)	1	159	22.8	19
Hindi–Bhojpuri aligned train	1	142	18.7	16
Aligned-data Dev (Bhojpuri)	2	87	17.2	15
BHTB gold test	3	67	16.4	14

The aligned-data and BHTB sentences are systematically shorter than HDTB sentences. This reflects the genres represented: HDTB draws from news and literature with longer formal sentences, while the aligned corpus and BHTB include more conversational and short-form texts. Sentence length affects parsing difficulty: longer sentences exhibit more long-range dependencies, which are systematically harder for biaffine parsers.

E.2 UPOS Tag Distributions

Two notable patterns. First, the four datasets agree closely on most tags, which is consistent with the shared linguistic typology of Hindi and Bhojpuri. Second, the `DET` frequency in the aligned corpus is notably higher than BHTB; this reflects an artefact of the auto-transfer pipeline, which sometimes labels Bhojpuri demonstrative pronouns as determiners rather than matching BHTB's conventions.

Table E.2: Most frequent UPOS tags by relative frequency (%) across the four datasets.

UPOS	HDTB train	Aligned train	Aligned Dev	BHTB
NOUN	26.4	23.9	24.1	25.3
VERB	11.2	12.1	11.7	11.6
ADJ	8.0	7.5	7.7	7.4
ADP	12.5	11.7	12.1	12.0
PRON	6.5	7.2	7.0	7.5
AUX	4.7	6.0	5.7	6.2
PUNCT	10.6	11.3	11.4	12.1
PROPN	8.1	6.4	6.3	6.7
NUM	3.0	2.8	2.7	2.9
ADV	3.5	3.7	3.6	3.8
CCONJ	2.4	2.1	2.2	2.0
SCONJ	1.0	1.2	1.2	1.1
DET	1.5	3.7	3.8	1.0

E.3 Dependency Relation Frequencies in BHTB

Table E.3: Top dependency relations by frequency in the BHTB gold test set (357 sentences, 5,856 tokens).

Relation	Count	Frequency (%)
case	907	15.5
nmod	907	15.5
punct	695	11.9
root	357	6.1
amod	202	3.4
nsubj	198	3.4
mark	123	2.1
obj	122	2.1
acl	105	1.8
advcl	82	1.4
xcomp	67	1.1
ccomp	58	1.0
cc	30	0.5

The `case` and `nmod` relations dominate, accounting for ∼31% of all tokens between them. This is a consequence of Hindi–Bhojpuri being heavy postposition languages: every nominal phrase typically has at least one postposition (`case`) and one nominal modifier (`nmod`). The clausal relations (`ccomp`, `xcomp`, `advcl`, `acl`) are relatively rare individually but collectively account for ∼5% of tokens—and these are precisely the relations that System A and System K struggle on, as shown in

the per-relation breakdown in Section 5.5.7.

E.4 Hindi–Bhojpuri Aligned Corpus Source Composition

Table E.4: Approximate domain composition of the Hindi–Bhojpuri aligned corpus.

Domain	**Approximate Share (%)**
Wikipedia (cultural / encyclopedic)	42
Newspaper articles	23
Literary translations	17
Government / institutional	11
Conversational / dialogue	4
Other	3

The corpus's domain skew toward written, formal text (Wikipedia, newspapers, literature) is consistent with the available digitised Bhojpuri sources. This genre bias affects model behaviour: parsers trained on this corpus will be strongest on formal written text and weaker on transcribed speech or social-media-style text.

E.5 Inter-Annotator Agreement Considerations

The BHTB gold test set was annotated by trained linguists following UD v2 guidelines, with cross-validation between annotators on a subset of sentences. While precise inter-annotator agreement (IAA) numbers are not published in the BHTB release notes, comparable Indic UD treebanks report Cohen's $\kappa = 0.85$–0.92 for UPOS tags and around 0.80 for dependency labels (per BHTB documentation). This places a soft upper bound on attainable LAS: even a perfect parser cannot exceed annotator agreement, so the practical ceiling for BHTB LAS is plausibly around 80%. Our System K result of 36.70% LAS, while notably below this ceiling, is consistent with the fundamental constraint of having no Bhojpuri gold training data.

Chapter F

Comparison with Published Indic UD Parsing Results

This appendix places our results in the context of published Indic-language parsing results.

F.1 Hindi Parsing Benchmarks

Table F.1: Published Hindi parsing results on HDTB UD v2.5.

System	Backbone	UAS	LAS	Year
UDPipe v2.5	word2vec	95.07	90.23	2020
Stanza v1.1.1	word2vec+BiLSTM	96.66	91.74	2020
Trankit base (XLM-R)	XLM-R-base	96.54	92.70	2021
Trankit large (XLM-R-L)	XLM-R-large	96.54	95.95	2021
System A (ours, reproduced)	XLM-R-base	∼95.6	∼92.6	2026

Our System A reproduces the published Trankit-base Hindi numbers within ±1 LAS, validating our pipeline. Hindi parsing is now mature: published LAS numbers cluster between 91 and 96, with the main differentiator being the backbone's pretraining scale (XLM-R-base vs. XLM-R-large).

F.2 Bhojpuri / Low-Resource Indic Parsing

The literature on Bhojpuri parsing is extremely limited. The BHTB release [Ojha and Zeman, 2022] presents preliminary baselines using cross-lingual transfer from Hindi at approximately 30% LAS—in the same range as our System A baseline. To our knowledge, no prior published work on Bhojpuri exceeds 35 LAS on BHTB, making our System K result of **36.70 LAS** a clear improvement on the public state-of-the-art.

For closely related low-resource Indic languages, the picture is similar: Ojha et al. [2019] report 25–40 LAS for cross-lingual Sanskrit parsing, and benchmarks for Magahi, Awadhi, and Maithili (where they exist) tend to fall in the 30–40 LAS range when only zero-shot or projection-based methods are available. Our methodology—adapter alignment for Part 1 and UD-Bridge silver self-training for Part 2—is broadly applicable to these settings.

F.3 Why Cross-Lingual UD Numbers Look Lower

Readers from outside the cross-lingual UD parsing community sometimes find the absolute numbers (35–40 LAS) surprisingly low compared to monolingual high-resource settings (90+ LAS). The gap reflects several intrinsic challenges:

1. **Zero gold target-language training.** Unlike monolingual setups where train and test data come from the same distribution, our setup has zero Bhojpuri gold training instances.

2. **Schema and distribution shift.** Even when training data is nominally in the same UD schema, the source-language treebank (HDTB) and the target-language test set (BHTB) differ in genre and domain.

3. **Small test set noise.** 357 sentences are not enough to smooth out per-relation noise, especially for rare clausal relations.

4. **Annotation idiosyncrasies.** BHTB follows strict UD v2 conventions, which may differ subtly from how a Hindi-trained parser was implicitly taught.

In this context, achieving **36.70 LAS** on BHTB through fully automatic methods (no human-annotated Bhojpuri training data) represents meaningful progress in low-resource Indic parsing.

Chapter G

Glossary of Key Terms

Adapter A small trainable bottleneck module inserted into a frozen pretrained transformer, used for parameter-efficient fine-tuning.

Aligned corpus / Aligned data Throughout this thesis, refers to the Hindi–Bhojpuri parallel corpus of 30,966 sentence pairs with token-level positional alignment and auto-transferred dependency annotations.

Arc A directed edge from a dependent token to its head token in a dependency tree.

Backbone The shared (frozen) pretrained transformer encoder underlying language-specific adapters.

Biaffine attention The bilinear scoring mechanism used to score head–dependent pairs in graph-based dependency parsers.

BHTB Bhojpuri Hindi Treebank: the 357-sentence gold UD test set for Bhojpuri parsing.

Catastrophic forgetting The phenomenon where a fine-tuned model loses its pre-trained general knowledge during task-specific training.

Cross-lingual transfer Training a model on one language and applying it to another, with or without target-language fine-tuning.

CTS Cross-lingual Tree Supervision: reusing source-language gold trees as additional supervision signal for the target language.

CoNLL-U The standard tab-separated text format for distributing UD annotated treebanks.

HDTB Hindi Dependency Treebank: the standard 13,304-sentence Hindi UD training treebank.

KL distillation Knowledge distillation using KL divergence between teacher and student probability distributions.

LAS Labelled Attachment Score: the fraction of tokens with both correct head and correct relation label. The standard primary metric for UD parsing.

MST Maximum Spanning Tree (algorithm); used to ensure tree validity when decoding biaffine arc scores.

Pfeiffer adapter A specific adapter variant where one bottleneck module is inserted after the feed-forward sublayer of each transformer block.

SACT Syntax-Aware Cross-lingual Transfer: the System H training framework combining content-word cosine alignment, arc-KL distillation, and cross-lingual tree supervision.

Schema mismatch A systematic difference between two annotation schemes (in this thesis: between auto-transferred labels and gold UD annotations) that causes evaluation gaps even for a high-quality parser.

Self-training Using a model's own predictions on unlabelled data as training signal for a subsequent model.

Silver labels Automatically generated annotations (as opposed to gold human annotations).

Trankit An open-source multilingual NLP pipeline built on XLM-R with language-specific adapters.

UAS Unlabelled Attachment Score: the fraction of tokens with the correct head, regardless of label.

UD Universal Dependencies: a cross-linguistically consistent annotation framework for morphology and dependency syntax.

UD-Bridge The System K training pipeline that uses System A's predictions as silver UD labels for Bhojpuri, then trains a new Trankit pipeline on Hindi gold + Bhojpuri silver data.

Warm-start fine-tuning Initialising fine-tuning from a previously trained checkpoint (e.g., the Hindi Trankit checkpoint), rather than from raw pretrained weights.

XLM-R / XLM-RoBERTa The 100-language multilingual masked language model from Meta AI used as the shared backbone in this thesis.

Bibliography

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.

Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. Unsupervised cross-lingual representation learning at scale. pages 8440–8451, 2020.

Atul Kr. Ojha and Daniel Zeman. Universal dependency treebanks for low-resource Indian languages: The case of Bhojpuri. In *Proceedings of WILDRE-6, LREC 2022*, pages 28–33, 2022.

Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Draft online, Stanford University, 3rd edition, 2023.

Pratik Joshi, Sebastin Santy, Amanul Budhiraja, Kalika Bali, and Monojit Choudhury. The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of ACL 2020*, pages 6282–6293, 2020.

Divyanshu Kakwani, Anoop Kunchukuttan, Satish Golla, N.C. Gokul, Avik Bhatt, Mitesh M. Khapra, and Pratyush Kumar. IndicNLPSuite: Monolingual corpora, evaluation benchmarks and pre-trained multilingual language models for indian languages. In *Findings of EMNLP 2020*, pages 4948–4961, 2020.

Lucien Tesnière. *Éléments de Syntaxe Structurale*. Klincksieck, Paris, 1959.

Igor A. Melčuk. *Dependency Syntax: Theory and Practice*. SUNY Press, Albany, NY, 1988.

Richard Hudson. *Language Networks: The New Word Grammar*. Oxford University Press, 2007.

Jan Hajič, Jarmila Panevová, Eva Hajicová, Petr Sgall, Petr Pajas, Jan Štěpánek, Jiří Havelka, Marie Mikulová, and Zdeněk Žabokrtský. Prague Dependency Treebank 2.0. In *LDC Catalogue*. Linguistic Data Consortium, 2006.

Hiroyasu Yamada and Yuji Matsumoto. Statistical dependency analysis with support vector machines. In *Proceedings of the 8th International Workshop on Parsing Technologies (IWPT)*, pages 195–206, 2003.

Joakim Nivre. An efficient algorithm for projective dependency parsing. In *Proceedings of the 8th International Workshop on Parsing Technologies (IWPT)*, pages 149–160, 2003.

Ryan McDonald, Fernando Pereira, Kiril Ribarov, and Jan Hajič. Non-projective dependency parsing using spanning tree algorithms. In *Proceedings of HLT/EMNLP 2005*, pages 523–530, 2005.

Ryan McDonald, Kevin Lerman, and Fernando Pereira. Multilingual dependency analysis with a two-stage discriminative parser. In *Proceedings of CoNLL-X*, pages 216–220, 2006.

Sabine Buchholz and Erwin Marsi. CoNLL-X shared task on multilingual dependency parsing. In *Proceedings of CoNLL-X*, pages 149–164, 2006.

Joakim Nivre, Johan Hall, Sandra Kübler, Ryan McDonald, Jens Nilsson, Sebastian Riedel, and Deniz Yuret. The CoNLL 2007 shared task on dependency parsing. In *Proceedings of the CoNLL Shared Task Session of EMNLP-CoNLL 2007*, pages 915–932, 2007.

Danqi Chen and Christopher D. Manning. A fast and accurate dependency parser using neural networks. In *Proceedings of EMNLP 2014*, pages 740–750, 2014.

Timothy Dozat and Christopher D. Manning. Deep biaffine attention for neural dependency parsing. In *Proceedings of ICLR 2017*, 2017a. URL `https://arxiv.org/abs/1611.01734`.

Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. Stanza: A Python natural language processing toolkit for many human languages. In *Proceedings of ACL 2020: System Demonstrations*, pages 101–108, 2020.

Minh Van Nguyen, Viet Dac Lai, Amir Pouran Ben Veyseh, and Thien Huu Nguyen. Trankit: A light-weight transformer-based toolkit for multilingual natural language processing. In *Proceedings of EACL 2021 (Demonstrations)*, pages 80–90, 2021.

Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Yoav Goldberg, Jan Hajič, Christopher D. Manning, Ryan McDonald, Slav Petrov, Sampo Pyysalo, Natalia Silveira, Reut Tsarfaty, and Daniel Zeman. Universal dependencies v1: A multilingual treebank collection. 2016.

Yoeng-Jin Chu and Tseng-Hong Liu. On the shortest arborescence of a directed graph. *Science Sinica*, 14:1396–1400, 1965.

Jack Edmonds. Optimum branchings. *Journal of Research of the National Bureau of Standards B*, 71B:233–240, 1967.

Neil Houlsby, Andrei Giurgiu, Stanisław Jastrzębski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *Proceedings of ICML 2019*, pages 2790–2799, 2019.

Jonas Pfeiffer, Andreas Rücklé, Clifton Poth, Aishwarya Kamath, Ivan Vulić, Sebastian Ruder, Kyunghyun Cho, and Iryna Gurevych. Adapterhub: A framework for adapting transformers. In *Proceedings of EMNLP 2020 (Demonstrations)*, pages 46–54, 2020a.

Jonas Pfeiffer, Ivan Vulić, Iryna Gurevych, and Sebastian Ruder. MAD-X: An adapter-based framework for multi-task cross-lingual transfer. In *Proceedings of EMNLP 2020*, pages 7654–7673, 2020b.

Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:109–165, 1989.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of NAACL-HLT 2006*, pages 152–159, 2006.

Terry Winograd. *Understanding Natural Language*. Academic Press, 1972.

John Lafferty, Andrew McCallum, and Fernando Pereira. Conditional random fields: Probabilistic models for segmenting and labeling sequence data. In *Proceedings of ICML 2001*, pages 282–289, 2001.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. pages 5998–6008, 2017.

Daniel Zeman, Martin Popel, Milan Straka, et al. CoNLL 2017 shared task: Multilingual parsing from raw text to universal dependencies. In *Proceedings of the CoNLL 2017 Shared Task*, 2017.

Daniel Zeman, Jan Hajič, Martin Popel, et al. CoNLL 2018 shared task: Multilingual parsing from raw text to universal dependencies. In *Proceedings of the CoNLL 2018 Shared Task*, 2018.

Ryan McDonald, Slav Petrov, and Keith Hall. Multi-source transfer of delexicalized dependency parsers. In *Proceedings of EMNLP 2011*, 2011.

Sebastian Schuster, Sonal Gupta, Rushin Shah, and Mike Lewis. Cross-lingual transfer learning for multilingual task oriented dialog. In *Proceedings of NAACL-HLT 2019*, pages 3795–3805, 2019.

Shijie Wu and Mark Dredze. Beto, Bentz, Becas: The surprising cross-lingual effectiveness of BERT. In *Proceedings of EMNLP-IJCNLP 2019*, pages 833–844, 2019.

Wasi Uddin Ahmad, Zhisong Zhang, Zuezhe Ma, Eduard Hovy, Kai-Wei Chang, and Nanyun Peng. On difficulties of cross-lingual transfer with order differences: A case study on dependency parsing. In *Proceedings of NAACL-HLT 2019*, pages 2440–2452, 2019.

Ahmet Üstün, Arianna Bisazza, Gosse Bouma, and Gertjan van Noord. UDapter: Language adaptation for truly universal dependency parsing. In *Proceedings of EMNLP 2020*, pages 2302–2315, 2020.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of ACL/IJCNLP 2021*, pages 3118–3135, 2021.

Oscar Täckström, Ryan McDonald, and Joakim Nivre. Token and type constraints for cross-lingual part-of-speech tagging. In *Transactions of the ACL*, volume 1, pages 1–12, 2013.

Mohammad Sadegh Rasooli and Michael Collins. Cross-lingual syntactic transfer with limited resources. In *Transactions of the ACL*, volume 5, pages 279–293, 2017.

Nafise Sadat Moosavi, William Held, Ivan Vulić, and Iryna Gurevych. Adapter-based finetuning for morphological analysis. In *Proceedings of NAACL-HLT 2021*, 2021.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL/IJCNLP 2021*, pages 4582–4597, 2021.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of EMNLP 2021*, pages 3045–3059, 2021.

Simran Khanuja, Divyanshu Bansal, Sarthak Mehra, Sonam Aggarwal, Sumit Pandey, Siddhant Garg, Sachin Kumar, Debayan Mahata, Parag Maheshwari, Hema Sharma, and Partha Talukdar. MuRIL: Multilingual representations for indian languages. 2021.

Atul Kr. Ojha, Pitambar Behera, Diwakar Mani Singh, Sarvjeet Sah, and Girish Nath Jha. Universal dependency treebanks for low-resource Indian languages: The case of Bhojpuri. In *Proceedings of the WILDRE-5 Workshop, LREC 2019*, 2019.

Mark Steedman, Miles Osborne, Anoop Sarkar, Stephen Clark, Rebecca Hwa, Julia Hockenmaier, Paul Ruhlen, Steven Baker, and Jeremiah Crim. Bootstrapping statistical parsers from small datasets. In *Proceedings of EACL 2003*, pages 331–338, 2003.

Kenji Sagae. Self-training without reranking for parser domain adaptation and its impact on semantic role labeling. *Proceedings of the 2010 Workshop on Domain Adaptation for NLP, ACL*, pages 37–44, 2010.

Sawan Kumar and Partha P. Talukdar. Reordering examples helps during priming-based few-shot learning. In *Findings of ACL 2021*, 2021.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

Adhiguna Kuncoro, Miguel Ballesteros, Lingpeng Kong, Chris Dyer, and Noah A. Smith. Distilling an ensemble of greedy dependency parsers into one MST parser. In *Proceedings of EMNLP 2016*, pages 1744–1753, 2016.

Xinyu Wang, Yong Jiang, Nguyen Bach, Tao Wang, Zhongqiang Huang, Fei Huang, and Kewei Tu. Structure-level knowledge distillation for multilingual sequence labeling. In *Proceedings of ACL 2020*, pages 3317–3330, 2020.

Yu-Hsiang Lin, Chian-Yu Chen, Jean Lee, Zirui Li, Yuyan Zhang, Mengzhou Xia, Shruti Rijhwani, Junxian He, Zhisong Zhang, Xuezhe Ma, Antonios Anastasopoulos, Patrick Littell, and Graham Neubig. Choosing transfer languages for cross-lingual learning. In *Proceedings of ACL 2020*, 2020.

Milan Gritta and Ignacio Iacobacci. XeroAlign: Zero-shot cross-lingual transformer alignment. In *Findings of ACL 2022*, 2022.

Tianyu Liu and Hany Hassan Awadalla. Cross-lingual self-training for low-resource nlp. In *Proceedings of NAACL 2022*, 2022.

Timothy Dozat and Christopher D. Manning. Deep biaffine attention for neural dependency parsing. *Proceedings of ICLR 2017*, 2017b.